

HASHING

Module - 6

HASHING

- ✓ Hashing is a technique or process of mapping keys, values into the hash table by using a hash function.
- ✓ It is done for faster access to elements.
- ✓ The efficiency of mapping depends on the efficiency of the hash function used.

WHAT IS MAPPING

- ✓ Mapping is the way of linking two things together, referred to as “Key” and “Value”.
 - ✓ Key is an Identifier to uniquely identify the Data(Entity).
 - ✓ Value is the Actual Entity (and its details) which we want to store.
- ✓ Example: Employee.
 - ✓ Key to identify Employee and his/her attributes is Employee ID
 - ✓ Values can be Employee name, Grade, designation etc.

WHAT IS MAPPING

- ✓ Another Example: Student.
- ✓ Each student can have different properties like his name, his address, etc. But to recognize him as a student, he must be given a unique identifier, like Roll Number.
 - ✓ Key for a Student = Roll Number and
 - ✓ Value for Student = Student details of Name, Address, Age, and so on.

WHAT IS MAPPING

- ✓ linking of all Employee details with an Employee ID can also be treated as the mapping of Employee properties to an Employee ID.
- ✓ linking of all student details with a Roll Number can also be treated as the mapping of Student properties to a roll Number.

Roll Number	Student <Name, Age, Address>
101	Anmol, 25, Haryana
104	Manoj, 26, New Delhi
106	Nitin, 24, Gurgaon

WHAT IS MAPPING

- ✓ To refer to a particular student, all we need to know is his/her roll number and same Employee ID for Employee.
- ✓ Another **advantage of mapping** is that the name/age/address can be the same across multiple Students or employees, but the Roll Number or Employee ID would be unique.
- ✓ mapping helps to link an easier/smaller “key” (here Roll Number & Emp ID) to a particular “value”(here Student’s & Employee properties).

BACK TO HASHING

- ✓ Hashing is a way where we can get an Integer value from any Key. This Key may or may not be an integer, but after hashing is performed, it will return an Integer value for any Key.
- ✓ Hashing is required as the Key which was given in the input can not be used as the Memory location to place this key value.
- ✓ Hashing performs a transformation on this Key, to return an Integer value which can be used as the memory address(or Index in the Data structure) to place this Key-Value at.

HASHING ON NON-INTEGRAL KEYS

- ✓ It takes any Entity(as a Key) as input, and if that key is not an Integer, we need to tell it some way to get an Integer value(N) from that Entity.
- ✓ Once we have got an initial Integral value (N), this Integer can't be used as the Memory address to place this Key-Value at. Now hashing converts this Integer to another Integer value which can be used as the Memory Address/Index to place this key-Value at.

HASHING ON NON-INTEGRAL KEYS

✓ Example 1:

- ✓ If Key=Roll Number and Value=Student,
- ✓ then Key = Integer itself, and
- ✓ hashing will run on this Roll Number
- ✓ to return the Location to place this Key-Value pair at.

HASHING ON NON-INTEGRAL KEYS

✓ Example 2:

- ✓ If Key=Address and Value=Student, and where Key is itself an Entity:
- ✓ Step 1: We define a way to get some integer value from this Key. e.g. for Address, the pin code makes sense.
- ✓ Step 2: Hashing takes Pin code as the Input and will return the Memory Location to place this Key-Value pair at.

HASHING IN DATA STRUCTURES

- ✓ To solve the problem of efficiently finding and storing data in an array.
- ✓ To get the Memory address or the Index in the given data structure, where we can store this Key-Value pair at.
- ✓ Hashing takes **Key as the Input** and **returns us the memory address**(or the Index in the Data structure) to place this Key-value pair at.

HASHING IN DATA STRUCTURES – HASH TABLE

- ✓ Hashing in data structure uses hash tables to store the key-value pairs.
- ✓ The Data Structures(e.g., Hash table) used to store the Key-Value are of limited size.
 - ✓ If we need to store 3 students with their Roll number as 302, 312, 342, then we need the array of size at least 343, so that these 3 students are placed 302, 312, 342 index of that table.
 - ✓ Improve the search efficiency but is a massive waste of space.

HASHING IN DATA STRUCTURES – HASH TABLE

- ✓ Instead of directly storing the keys in the hash table, hashing comes into play, which runs the hash function converts these big Keys to the Integer value $\leq$ size of Array.
- ✓ These integer values are Hash Index.

HASHING IN DATA STRUCTURES – HASH FUNCTION

- ✓ Hash Function is a mathematical formula that can take Key as an input and can return some integer value, which would be $\leq$ size of Array.
- ✓ The hash function converts the Key into a small integer or hash Index. This integer is used as an index to store the original data.

HASHING IN DATA STRUCTURES – HASH FUNCTION

✓ Example:

Roll Number	Student <Name, Age, Address>
101	Anmol, 25, Haryana
104	Manoj, 26, New Delhi
106	Nitin, 24, Gurgaon

- ✓ To store three students according to roll no. we require 106 sized array which is big array to store only 3 values. In order to save space, we can just perform hash function and generate hash index using Array of size=3.
- ✓ Hash Function can be - $\text{RollNumber} \% \text{ArraySize}$.
 - ✓ $101 \% 3 = 2$ – place this student at index 2
 - ✓ $104 \% 3 = 2$ – Assume for time being index is 0
 - ✓ $106 \% 3 = 1$ - place this student at index 1

HASHING IN DATA STRUCTURES – HASH FUNCTION

- ✓ Hash function returns the same index for 2 Roll Numbers?
 - ✓ It's called Collision
 - ✓ Need to avoid collisions, so the hash function should be good enough that it gives distinct Indexes for every Key.
- ✓ Given some Key-Value, to store it we would need to:
 - ✓ Pass this Key to the hash function to get a unique index in the Array.
 - ✓ Store the Key inside the hash table, against this index.

COLLISION RESOLUTION TECHNIQUE

- ✓ Same index can be obtained by running the Hash Function on different keys.
 - ✓ If using the Division method: Let's say the array size=10
 - ✓ Then $\text{Hash}(15) = 15\%10 = 5$
 - ✓ And $\text{Hash}(25) = 25\%10 = 5$
 - ✓ So, this hash function will return 5 as the index for both of these keys.
- ✓ This is called a Collision, where the resultant hashes of 2 or more Keys, wrongly map to the same place in the hash table.

- ✓ Open addressing means where we try to resolve the collision by finding the alternative index to place the Key at.
 - ✓ Linear Probing
 - ✓ Quadratic probing
 - ✓ Rehashing

COLLISION RESOLUTION TECHNIQUE - OPEN ADDRESSING

✓ Linear Probing

- ✓ When the collision occurs, using the Linear Probing technique, the closest free cell is found by linearly iterating over the cells.
- ✓ The searching is performed sequentially, starting from the position where the collision occurs till the empty cell is not found.

COLLISION RESOLUTION TECHNIQUE - OPEN ADDRESSING

✓ Linear Probing – Example

Given - array of size 10 and choose a simple hash function

$\text{hash}(K) = (2K + 1) \% \text{size}$.

Input=2, $\text{hash}(2) = (2*2+1) \% 10 = 5 \% 10 = 5$

Input=7, $\text{hash}(7) = (2*7+1) \% 10 = 15 \% 10 = 5$ - Collision Occurred

linear probing is applied.

The nearest empty cell to the index 5 is 6; therefore, the Key=7 will be added at the index 6.

COLLISION RESOLUTION TECHNIQUE - OPEN ADDRESSING

✓ Linear Probing – Example

Given - array of size 10 and choose a simple hash function $\text{hash}(K) = (2K + 1) \% \text{size}$.

Input=12, $\text{hash}(12) = (2 \cdot 12 + 1) \% 10 = 25 \% 10 = 5$

As the 5th Index is again occupied with Key=2, we can't place 12 at 5, so we will again need to linearly probe from 5.

Similarly, the 6th index is also occupied with Key=7, we look for the next Free Index.

The 7th index is free, so we can place Key=12 at the 7th Index.

COLLISION RESOLUTION TECHNIQUE - OPEN ADDRESSING

✓ Linear Probing – Example

Given - array of size 10 and choose a simple hash function
 $\text{hash}(K) = (2K + 1) \% \text{size}$.

Array size = 10

Input = 2

$$\text{hash}(2) = (2 \times 2 + 1) \% 10 = 5$$

Input 2 = 7

$$\text{hash}(7) = (7 \times 2 + 1) \% 10 = 5$$

7 at 6th Index

6
1
2
3
4
5

COLLISION RESOLUTION TECHNIQUE - OPEN ADDRESSING

✓ **Rehashing**

- ✓ It invokes 2 hash functions to get rid of collisions.
- ✓ These 2 hash functions can either be the same or different and
- ✓ The secondary hash function is applied continuously until an unused/vacant index is found.

COLLISION RESOLUTION TECHNIQUE - OPEN ADDRESSING

✓ Rehashing - Example

Array size = 10.

Key=15

$\text{Hash1}(15) = 15\%10 = 5$

So, 15 is placed at the 5th Index.

Key=25

$\text{Hash1}(25) = 25\%10 = 5$

Since 5 is already used, it's a collision.

$\text{Hash2}(25) = 625$. Picking 1 middle element we get 2. So, 25 can be placed at the 2nd Index.

So, as we can see, by using 2 different hash functions, we are able to place both 15 and 25 at the 5th and 2nd index, respectively.

COLLISION RESOLUTION TECHNIQUE - OPEN ADDRESSING

✓ Rehashing – Example

Array size = 10

Input = 2

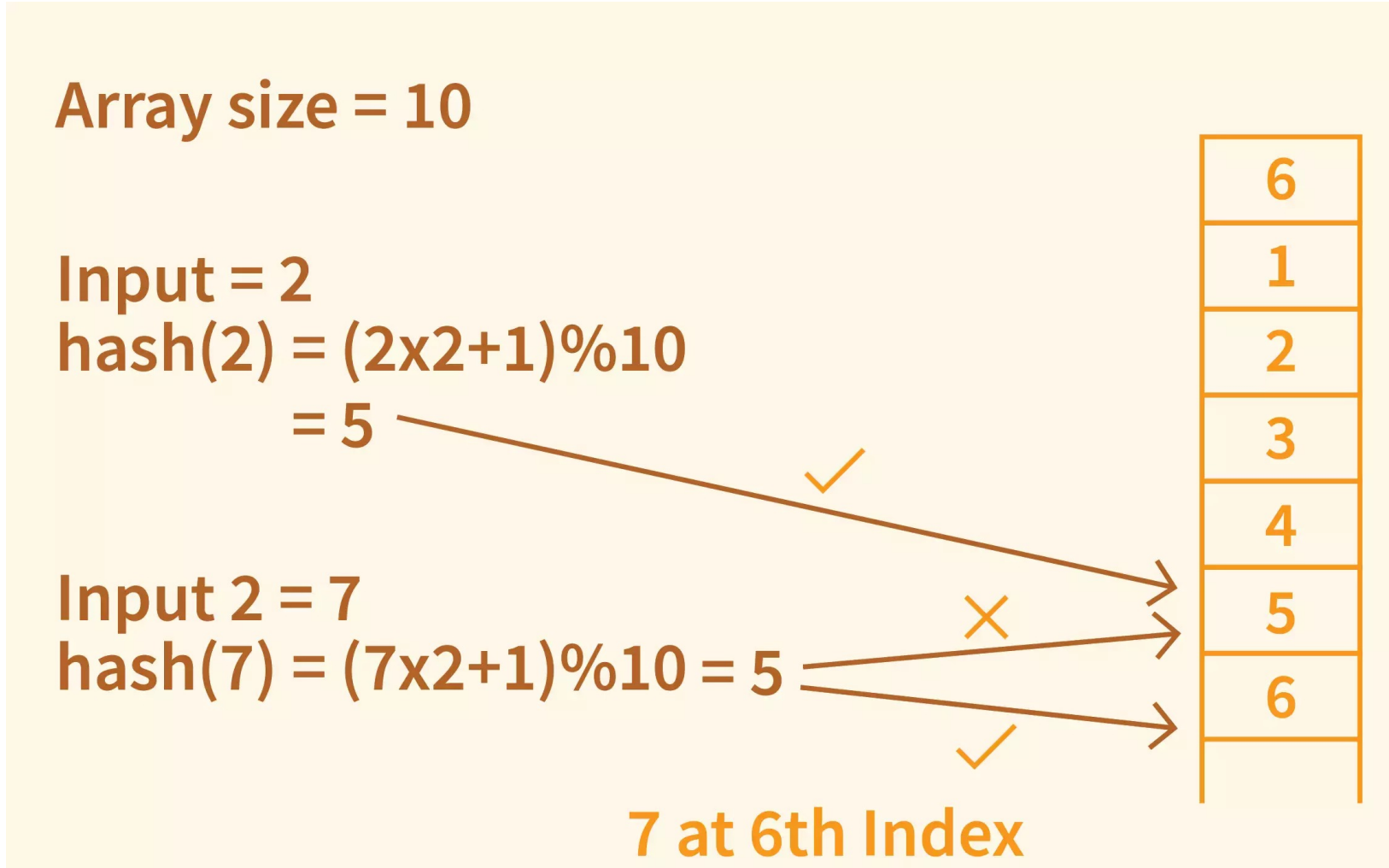
$$\text{hash}(2) = (2 \times 2 + 1) \% 10 \\ = 5$$

Input 2 = 7

$$\text{hash}(7) = (7 \times 2 + 1) \% 10 = 5$$

7 at 6th Index

6
1
2
3
4
5
6



COLLISION RESOLUTION TECHNIQUE - OPEN ADDRESSING

✓ Quadratic probing

- ✓ Like linear probing, but instead of moving one step at a time to find the empty cell to place this Data at, here the next slot is computed by adding the successive value of an arbitrary polynomial in the original hashed index.

COLLISION RESOLUTION TECHNIQUE - OPEN ADDRESSING

✓ Quadratic probing – Example

We have hash table with 10 slots and the following keys to be inserted: 15, 29, 12, 8, and 18.

Use the hash function $\text{hash}(\text{key}) = \text{key} \% 10$ to map the keys to their respective indices.

COLLISION RESOLUTION TECHNIQUE - OPEN ADDRESSING

✓ Quadratic probing – Example

Initial state of the hash table:

VALUE										
INDEX	0	1	2	3	4	5	6	7	8	9

Inserting key 15:

Calculate the index using the hash function: $\text{index} = 15 \% 10 = 5$.

The slot at index 5 is empty, so we insert 15 there.

VALUE						15				
INDEX	0	1	2	3	4	5	6	7	8	9

COLLISION RESOLUTION TECHNIQUE - OPEN ADDRESSING

✓ Quadratic probing – Example

Inserting key 29:

Calculate the index using the hash function: $\text{index} = 29 \% 10 = 9$.

The slot at index 9 is empty, so we insert 29 there.

Updated hash table:

VALUE						15				29
INDEX	0	1	2	3	4	5	6	7	8	9

COLLISION RESOLUTION TECHNIQUE - OPEN ADDRESSING

✓ Quadratic probing – Example

Inserting key 12:

Calculate the index using the hash function: $\text{index} = 12 \% 10 = 2$.

The slot at index 2 is empty so storing key 12.

VALUE			12			15				29
INDEX	0	1	2	3	4	5	6	7	8	9

COLLISION RESOLUTION TECHNIQUE - OPEN ADDRESSING

✓ Quadratic probing – Example

Inserting key 8:

Calculate the index using the hash function: $\text{index} = 8 \% 10 = 8$.

The slot at index 8 is empty, so we insert 8 there.

VALUE			12			15			8	29
INDEX	0	1	2	3	4	5	6	7	8	9

COLLISION RESOLUTION TECHNIQUE - OPEN ADDRESSING

✓ Quadratic probing – Example

Inserting key 18:

Calculate the index using the hash function: $\text{index} = 18 \% 10 = 8$.

The slot at index 8 is already occupied by key 8.

Since there is a collision, we use quadratic probing:

First quadratic probing: $\text{index} = (8 + 1^2) \% 10 = 9$.

The slot at index 9 is occupied by key 29.

Second quadratic probing: $\text{index} = (8 + 2^2) \% 10 = 4$.

The slot at index 4 is empty, so we insert 18 there.

VALUE			12		18	15		18	8	29
INDEX	0	1	2	3	4	5	6	7	8	9

COLLISION RESOLUTION TECHNIQUE - CLOSED ADDRESSING

- ✓ Place Values at that index only using the Chaining technique.
 - ✓ **Chaining**
 - ✓ The chaining method builds a Linked list of items whose key hashes to the same value.
 - ✓ It requires an extra link field to each table position.
 - ✓ If compared it against Rehashing, it only uses 1 hash function, but how the Data is stored is different.

COLLISION RESOLUTION TECHNIQUE - CLOSED ADDRESSING

✓ Place Values at that index only using the Chaining technique.

✓ **Chaining - Example**

Array size = 10.

Key=15

$\text{Hash}(15) = 15\%10 = 5$

So 15 is placed at the 5th Index.

Key=25

$\text{Hash}(25) = 25\%10 = 5$

Since 5 is already used, it's a collision.

COLLISION RESOLUTION TECHNIQUE - CLOSED ADDRESSING

- ✓ Place Values at that index only using the Chaining technique.
- ✓ **Chaining – Example**
- ✓ Chaining mechanism can place both 15 and 25 at the same 5th Index, by chaining 15 to 25.

Array size = 10

Input = 15
 $\text{hash}(15) = 15\%10 = 5$

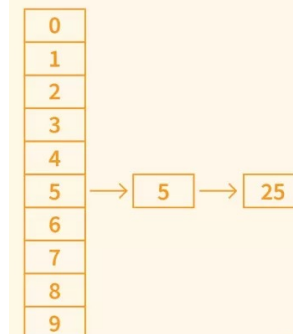


0	1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---	---

Input 2 = 25
 $\text{hash}(25) = 25\%10 = 5$

✗ Cant go

So we can chain:



COLLISION RESOLUTION TECHNIQUE - CLOSED ADDRESSING

- ✓ To retrieve the data in such cases, e.g. here we have 15 and 25 chained at Index=5.
- ✓ Now if any of 15 and 25 is given as input to search, we first calculate the hash to arrive at Index 5.
- ✓ Now since we have a chain, now we iterate over this chain until the Key matches.

COLLISION RESOLUTION TECHNIQUE - CLOSED ADDRESSING

- ✓ To store the Key-Value, we calculate the Hash of the key and place it at that index.
- ✓ Incase of a Collision, we chain to the existing element at that Index.
- ✓ To search back, we again calculate the hash to arrive at the index that(along with other chain of elements, perhaps) is stored.
- ✓ Iterate over the chain, comparing keys, to find the Element we are interested in.

COLLISION RESOLUTION TECHNIQUE – LINEAR PROBING

- ✓ Consider a hash table with 10 slots and the following keys to be inserted: 12, 25, 7, 18, 29, 15.

Use the hash function $\text{hash}(\text{key}) = \text{key} \% 10$ to determine the index for each key.

COLLISION RESOLUTION TECHNIQUE – LINEAR PROBING

- ✓ Initial state of the hash table:

VALUE										
INDEX	0	1	2	3	4	5	6	7	8	9

- ✓ Inserting key 12:

- ✓ Calculate the index using the hash function: $\text{index} = 12 \% 10 = 2$.
- ✓ The slot at index 2 is empty, so we insert 12 there.

VALUE			12							
INDEX	0	1	2	3	4	5	6	7	8	9

COLLISION RESOLUTION TECHNIQUE – LINEAR PROBING

- ✓ Inserting key 25:
 - ✓ Calculate the index using the hash function: $\text{index} = 25 \% 10 = 5$.
 - ✓ The slot at index 5 is empty, so we insert 25 there.

VALUE			12			25				
INDEX	0	1	2	3	4	5	6	7	8	9

COLLISION RESOLUTION TECHNIQUE – LINEAR PROBING

- ✓ Inserting key 7:
 - ✓ Calculate the index using the hash function: $\text{index} = 7 \% 10 = 7$.
 - ✓ The slot at index 7 is empty, so we insert 7 there.

VALUE			12			25		7		
INDEX	0	1	2	3	4	5	6	7	8	9

COLLISION RESOLUTION TECHNIQUE – LINEAR PROBING

✓ Inserting key 18:

- ✓ Calculate the index using the hash function: $\text{index} = 18 \% 10 = 8$.
- ✓ The slot at index 8 is empty, so we insert 18 there.

VALUE			12			25		7	18	
INDEX	0	1	2	3	4	5	6	7	8	9

COLLISION RESOLUTION TECHNIQUE – LINEAR PROBING

✓ Inserting key 29:

- ✓ Calculate the index using the hash function: $\text{index} = 29 \% 10 = 9$.
- ✓ The slot at index 9 is empty, so we insert 29 there.

VALUE			12			25		7	18	29
INDEX	0	1	2	3	4	5	6	7	8	9

COLLISION RESOLUTION TECHNIQUE – LINEAR PROBING

✓ Inserting key 15:

- ✓ Calculate the index using the hash function: $\text{index} = 15 \% 10 = 5$.
- ✓ The slot at index 5 is not empty, so we insert 15 at index 6.

VALUE			12			25	15	7	18	29
INDEX	0	1	2	3	4	5	6	7	8	9

REHASHING

- ✓ It is a process of increasing the size of a hashmap/Hash Table and redistributing the elements to new buckets based on their new hash values.
- ✓ To improve the performance of the hashmap and to prevent collisions caused by a high load factor.
- ✓ When a hashmap becomes full, the load factor increases. As the load factor increases, the number of collisions also increases, which can lead to poor performance.

REHASHING

- ✓ Load Factor - the ratio of the number of elements to the number of buckets.
- ✓ The hashmap can be resized and the elements can be rehashed to new buckets, which decreases the load factor and reduces the number of collisions.
- ✓ During rehashing, all elements of the hashmap are iterated and their new bucket positions are calculated using the new hash function that corresponds to the new size of the hashmap.

REHASHING – LOAD FACTOR

- ✓ Load factor = m/n
- ✓ where n is the total size of the hash table and
- ✓ m is the preferred number of entries which can be inserted before an increment in size of the underlying data structure is required.

REHASHING – DEFINITION

- ✓ Rehashing is a technique in which the table is resized, i.e., the size of table is doubled by creating a new table.
- ✓ the Load-Factor decides when we should increase the size of this data structure.

REHASHING

- ✓ The Hash table provides Constant time complexity of insertion and searching, provided the hash function can distribute the input load evenly.
- ✓ in case of Collision, the time complexity can go up to $O(N)$ in the worst case.
- ✓ Idea: Hash Table should store elements with as little collision as possible.

REHASHING

- ✓ Even with the best possible Hash function and with more and more elements coming to be stored inside this Hash table.
- ✓ Eventually we will run out of vacant indexes and will result in unavoidable collisions.
- ✓ To handle such a scenario we have the **Load Factor**.
- ✓ **load factor** is a measure that decides when exactly to increase the size of the Hash Table to maintain the same time complexity of $O(1)$.

REHASHING - LOAD FACTOR - EXAMPLE

- ✓ Given : HashTable of size=16:
 - ✓ If there are 16 elements in the HashTable, the hash function method will distribute one element in each Index.
 - ✓ If there are 32 elements in the HashTable, the hash function method will distribute two elements in each Index.
 - ✓ if there are 128 elements in HashTable, the hash function method will distribute eight elements in each Index.
 - ✓ the lookup also increases.

REHASHING - LOAD FACTOR - EXAMPLE

✓ Initial Capacity:

- ✓ The initial capacity is the number of Indexes allocated in the HashTable. It is created when the HashTable is initialized.
- ✓ The capacity of the HashTable is doubled each time it reaches the threshold.
- ✓ The default load factor of HashMap used in java, for instance, is 0.75f
- ✓ That means if we have a HashTable with an array size of 100, then whenever we have 75 elements stored, we will increase the size of the array to double of its previous size i.e. to 200 now, in this case.

REHASHING - LOAD FACTOR - USAGE

- ✓ The load factor can be decided using the following formula:

$$\text{Initial capacity of the HashTable} * \text{Load factor of the HashTable}$$

If The initial capacity of HashTable is = 16

And the load factor of HashTable = 0.75

According to the formula as mentioned above: $16 * 0.75 = 12$

REHASHING - LOAD FACTOR - USAGE

- ✓ According to the formula as mentioned above: $16 * 0.75 = 12$
- ✓ It represents that the 12th key-value pair of a HashTable will keep its size to 16.
- ✓ As soon as the 13th element (key-value pair) will come into the HashTable, it will increase its size from 16 buckets to $16 * 2 = 32$ buckets.

REHASHING - LOAD FACTOR – USAGE – ANOTHER METHOD

- ✓ When the current load factor becomes equal to the defined load factor i.e., 0.75, hashmap increases its capacity.
- ✓ Where:
 - ✓ m – is the number of entries in a HashTable
 - ✓ n – is the total size of HashTable

REHASHING - LOAD FACTOR – USAGE – ANOTHER METHOD

Example: If we have the initial capacity of HashTable = 16.

We insert the first element, now check if we need to increase the size of the HashTable capacity or not.

It can be determined by the formula: **Size of hashmap (m) / number of buckets (n)**

In this case, the size of the hashmap is 1, and the bucket size is 16. So, $1/16=0.0625$. Now compare this value with the default load factor. $0.0625 < 0.75$

So, no need to increase the hashmap size.

We do not need to increase the size of hashmap up to 12th element, because $12/16=0.75$

As soon as we insert the 13th element in the hashmap, the size of hashmap is increased because: $13/16=0.8125$

Which is greater than the default hashmap size. Now we need to increase the hashmap size

REHASHING

- ✓ Rehashing means hashing again
- ✓ As the size of the array is increased(usually doubled) and all the values are hashed again and stored in the new double sized array to maintain a low load factor and low complexity.
- ✓ if we had Array of size 100 earlier, and once we have stored 75 elements into it(given it has Load Factor=75), then when we need to store the 76th element, we double its size to 200.
- ✓ With the new size the Hash function can change, which means all the 75 elements we had stored earlier, would now with this new hash Function might yield different Index to place them, so basically we rehash all those stored elements with the new Hash Function and place them at new Indexes of newly resized bigger HashTable.

REHASHING - EXAMPLE

Given : HashTable with Initial Capacity of 4.

Insert 4 Keys: 100, 101, 102, 103.

Use hash function: $\text{Key} \% \text{ArraySize}$

So $\text{Hash}(100) = 0$, so Element stored at 1st Index.

So $\text{Hash}(101) = 1$, so Element stored at 2nd Index.

So $\text{Hash}(102) = 2$, so Element stored at 3rd Index.

With the insertion of 3 elements, the load on Hash Table = $\frac{3}{4} = 0.75$

REHASHING - EXAMPLE

Since the size is reaching 75% the size of the hash table will be doubled.

Now Hash table is of size – 8.

Perform rehashing of all the keys again with the hash function as $k \bmod n$ where n is the new size of hash table.

EXTENDIBLE HASHING

- ✓ Used to efficiently organize and access data in a hash table.
- ✓ It is designed to handle dynamic data sets, where the number of keys and the amount of data can vary over time.

EXTENDIBLE HASHING

- ✓ A hash table is divided into a fixed number of buckets or blocks.
- ✓ Each bucket can store a certain number of key-value pairs.
- ✓ Initially, the hash table starts with a small number of buckets, typically with just one block.
- ✓ As data is inserted into the hash table, it is hashed using a hash function to determine the bucket where it should be stored.

EXTENDIBLE HASHING – KEY IDEA

- ✓ The key idea behind extendible hashing is that
 - ✓ Use a directory to store addresses of the buckets in pointers.
 - ✓ An id is assigned to each directory which may change each time when Directory Expansion takes place.
 - ✓ The directories store pointers to buckets, and the buckets are used to hash the actual data.
 - ✓ The hash function returns a directory id which is used to navigate to the appropriate bucket.
 - ✓ This allows for dynamic changes in the hash function and an incremental approach to re-hashing

EXTENDIBLE HASHING – KEY IDEA

- ✓ Local depth and Global depth are associated with the directories and buckets used to hash data.
- ✓ **Directories:**
 - ✓ The directories store addresses of the buckets in pointers.
 - ✓ An id is assigned to each directory which may change each time when Directory Expansion takes place.
- ✓ Global depth is associated with the directories and denotes the number of bits used by the hash function to categorize the keys. The number of directories is equal to **2 raised to the power of the global depth.**

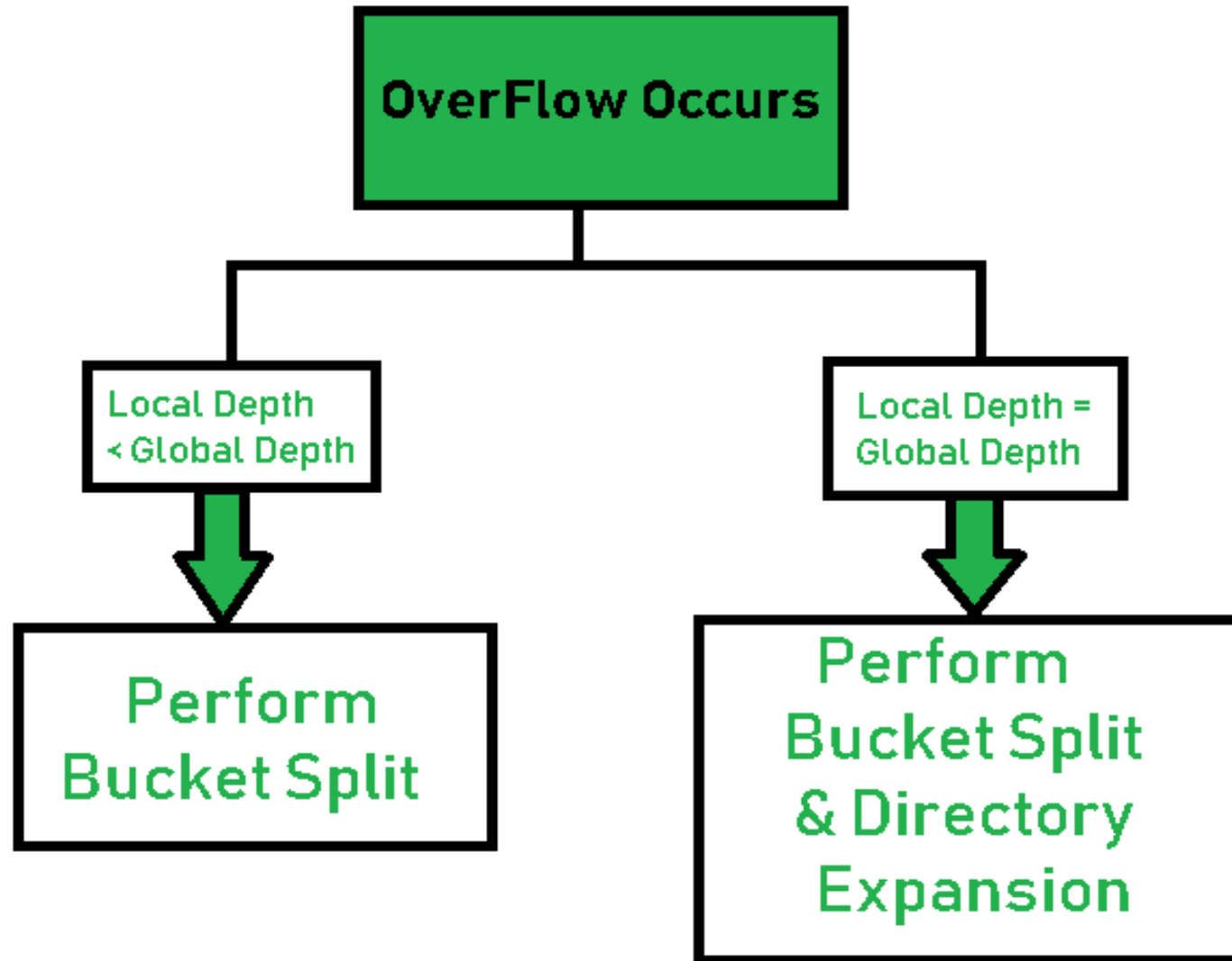
EXTENDIBLE HASHING – KEY IDEA

- ✓ **Buckets:** The buckets are used to hash the actual data.
- ✓ **Local depth** is associated with the buckets and is used to decide the action that needs to be performed in case an overflow occurs. Local depth is always less than or equal to the global depth.

EXTENDIBLE HASHING – WORKING

- ✓ **Analyze Data Elements:** Data elements may exist in various forms such as Integer, String, Float, etc.
- ✓ **Convert into binary format:** Convert the data element into Binary form. For string elements, consider the ASCII equivalent integer of the starting character and then convert the integer into binary form.
- ✓ **Check Global Depth of the directory:** Suppose the global depth of the Hash-directory is 3.
- ✓ **Identify the Directory:** Consider the 'Global-Depth' number of Least Significant Bits (LSBs) in the binary number and match it to the directory id.
- ✓ **Navigation:** Now, navigate to the bucket pointed by the directory with the matched directory-id.
- ✓ **Insertion and Overflow Check:** Insert the element and check if the bucket overflows. If a bucket overflows, it can be split into two parts. If the local depth of the overflowing bucket is equal to the global depth, then a Directory Expansion takes place.

EXTENDIBLE HASHING – WORKING



EXTENDIBLE HASHING – EXAMPLE

- ✓ Consider the following elements: 16,4,6,22,24,10,31,7,9,20,26.
Bucket Size: 3 (Assume)
- ✓ Hash Function: Suppose the global depth is X . Then the Hash Function returns X LSBs.

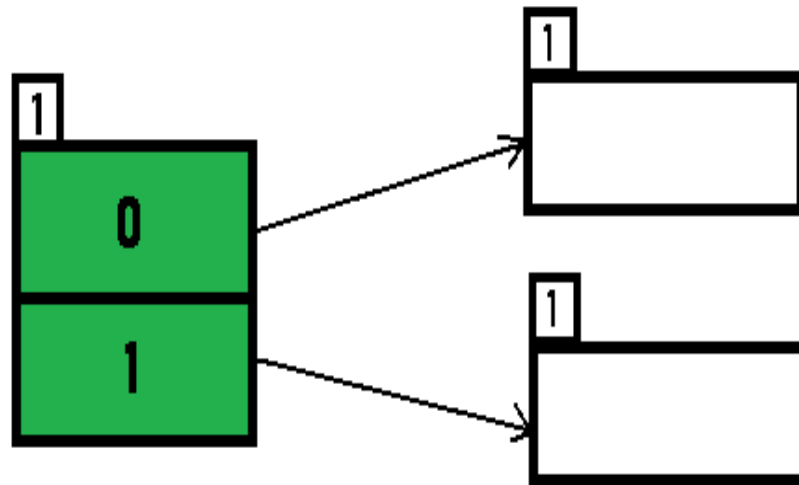
EXTENDIBLE HASHING – EXAMPLE

- ✓ Step 1 - calculate the binary forms of each of the given numbers.

16 - 10000	10 - 01010
4 - 00100	31 - 11111
6 - 00110	7 - 00111
22 - 10110	9 - 01001
24 - 11000	20 - 10100
	26 - 11010

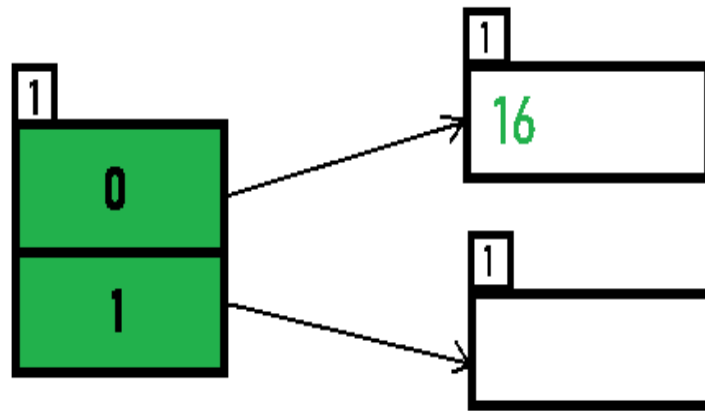
EXTENDIBLE HASHING – EXAMPLE

- ✓ Initially, the global-depth and local-depth is always 1. Thus, the hashing frame looks like this:



EXTENDIBLE HASHING – EXAMPLE

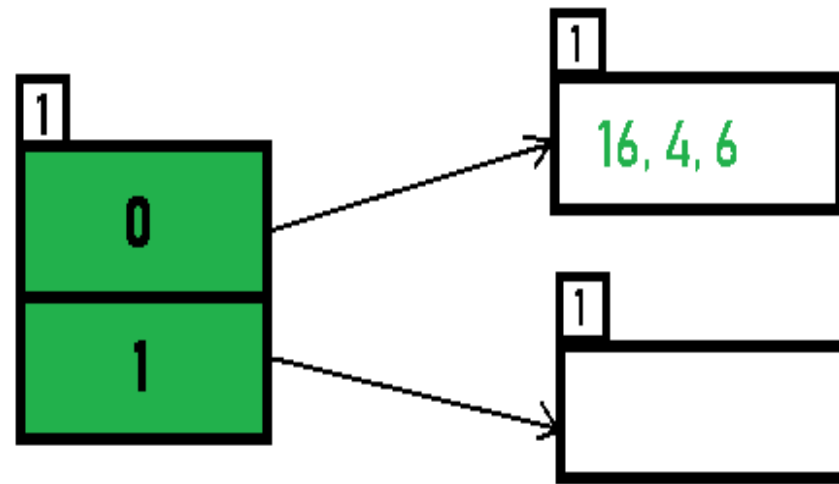
- ✓ Inserting 16: The binary format of 16 is 10000 and global-depth is 1. The hash function returns 1 LSB of 10000 which is 0. Hence, 16 is mapped to the directory with id=0.



$\text{Hash}(16) = 1000\underline{0}$

EXTENDIBLE HASHING – EXAMPLE

- ✓ Inserting 4 and 6: Both 4(100) and 6(110) have 0 in their LSB. Hence, they are hashed as follows:



$Hash(4)=10\underline{0}$

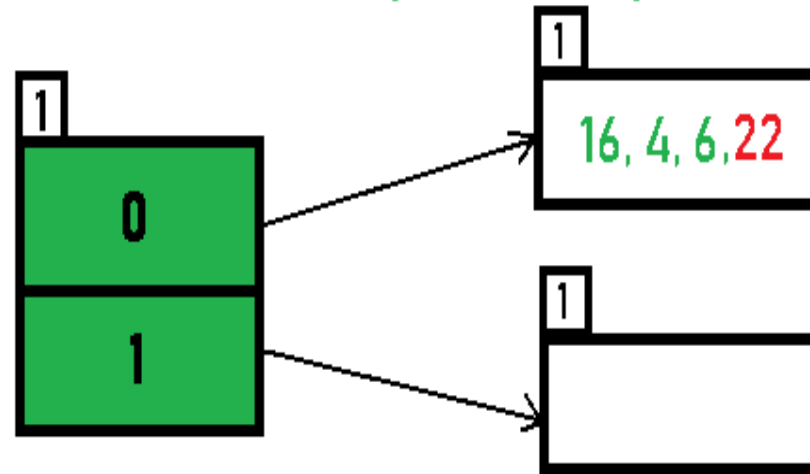
$Hash(6)=11\underline{0}$

EXTENDIBLE HASHING – EXAMPLE

- ✓ Inserting 22: The binary form of 22 is 10110. Its LSB is 0. The bucket pointed by directory 0 is already full. Hence, Overflow occurs.

OverFlow Condition

Here, Local Depth=Global Depth



Hash(22)=10110

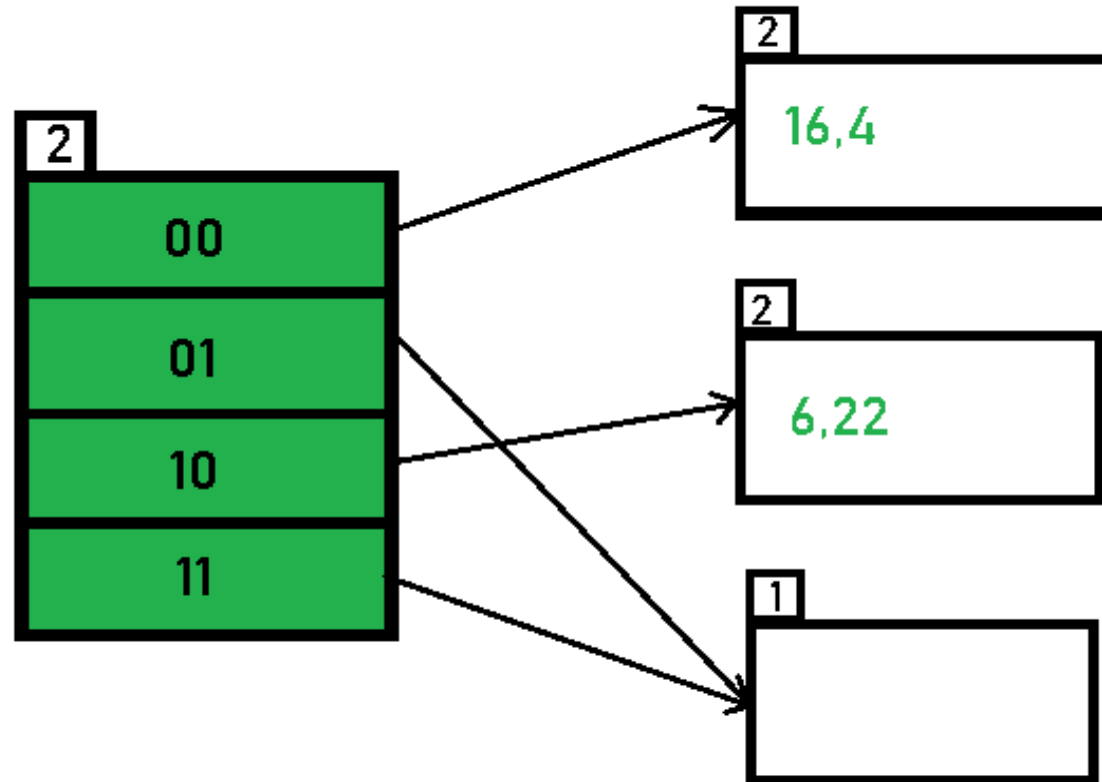
EXTENDIBLE HASHING – EXAMPLE

- ✓ **Case 1**, Since Local Depth = Global Depth, the bucket splits and directory expansion takes place.
- ✓ Rehashing of numbers present in the overflowing bucket takes place after the split.
- ✓ Since the global depth is incremented by 1, now, the global depth is 2.
- ✓ Hence, 16,4,6,22 are now rehashed w.r.t 2

LSBs.[16(10000),4(100),6(110),22(10110)]

EXTENDIBLE HASHING – EXAMPLE

After Bucket Split and Directory Expansion

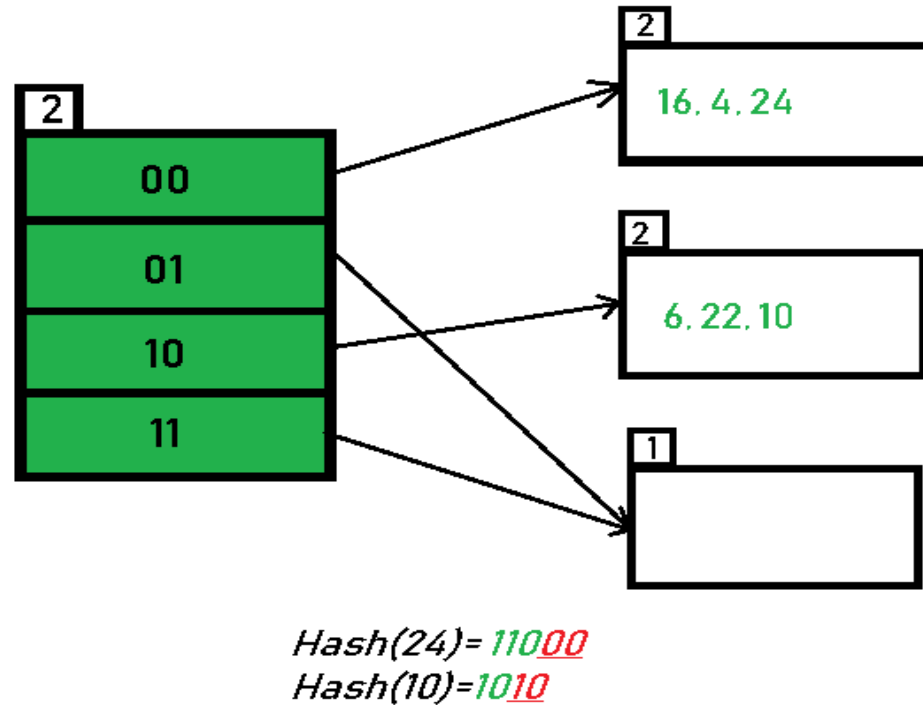


EXTENDIBLE HASHING – EXAMPLE

- ✓ Notice that the bucket which was underflow has remained untouched.
- ✓ But, since the number of directories has doubled, we now have 2 directories 01 and 11 pointing to the same bucket.
- ✓ This is because the local-depth of the bucket has remained 1.
- ✓ Any bucket having a local depth less than the global depth is pointed-to by more than one directories.

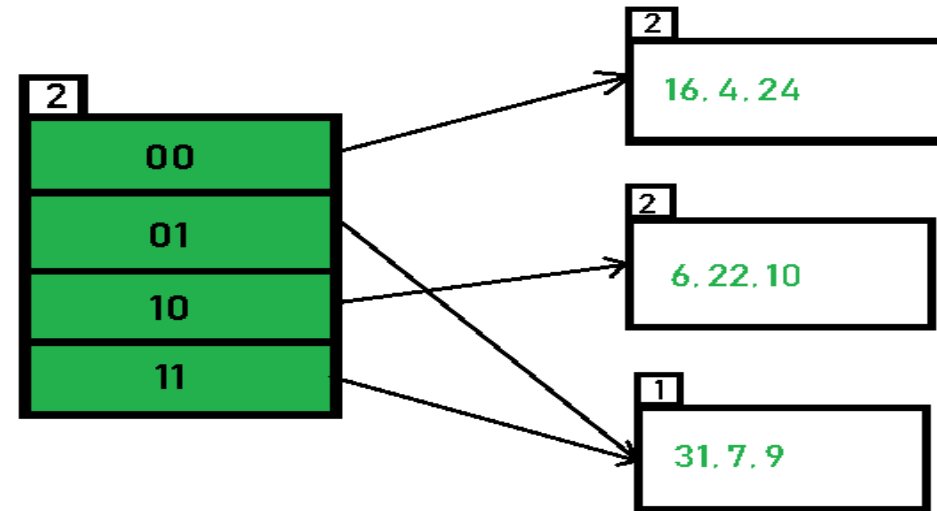
EXTENDIBLE HASHING – EXAMPLE

- ✓ Inserting 24 and 10: 24(11000) and 10 (1010) can be hashed based on directories with id 00 and 10. Here, we encounter no overflow condition.



EXTENDIBLE HASHING – EXAMPLE

- ✓ Inserting 31,7,9: All of these elements[31(111**11**), 7(1**11**), 9(10**01**)] have either 01 or 11 in their LSBs. Hence, they are mapped on the bucket pointed out by 01 and 11. We do not encounter any overflow condition here.

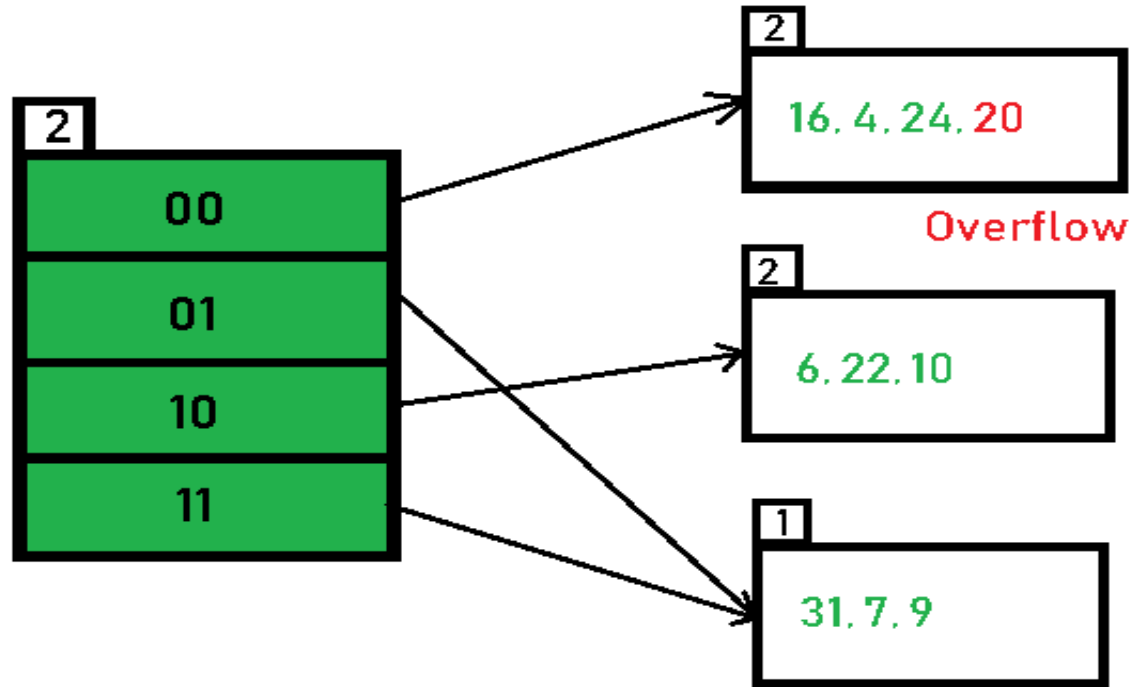


$Hash(31) = 11111$
 $Hash(7) = 111$
 $Hash(9) = 1001$

EXTENDIBLE HASHING – EXAMPLE

- ✓ Inserting 20: Insertion of data element 20 (10100) will again cause the overflow problem.

Overflow, Local Depth = Global Depth



Hash(20) = 10100

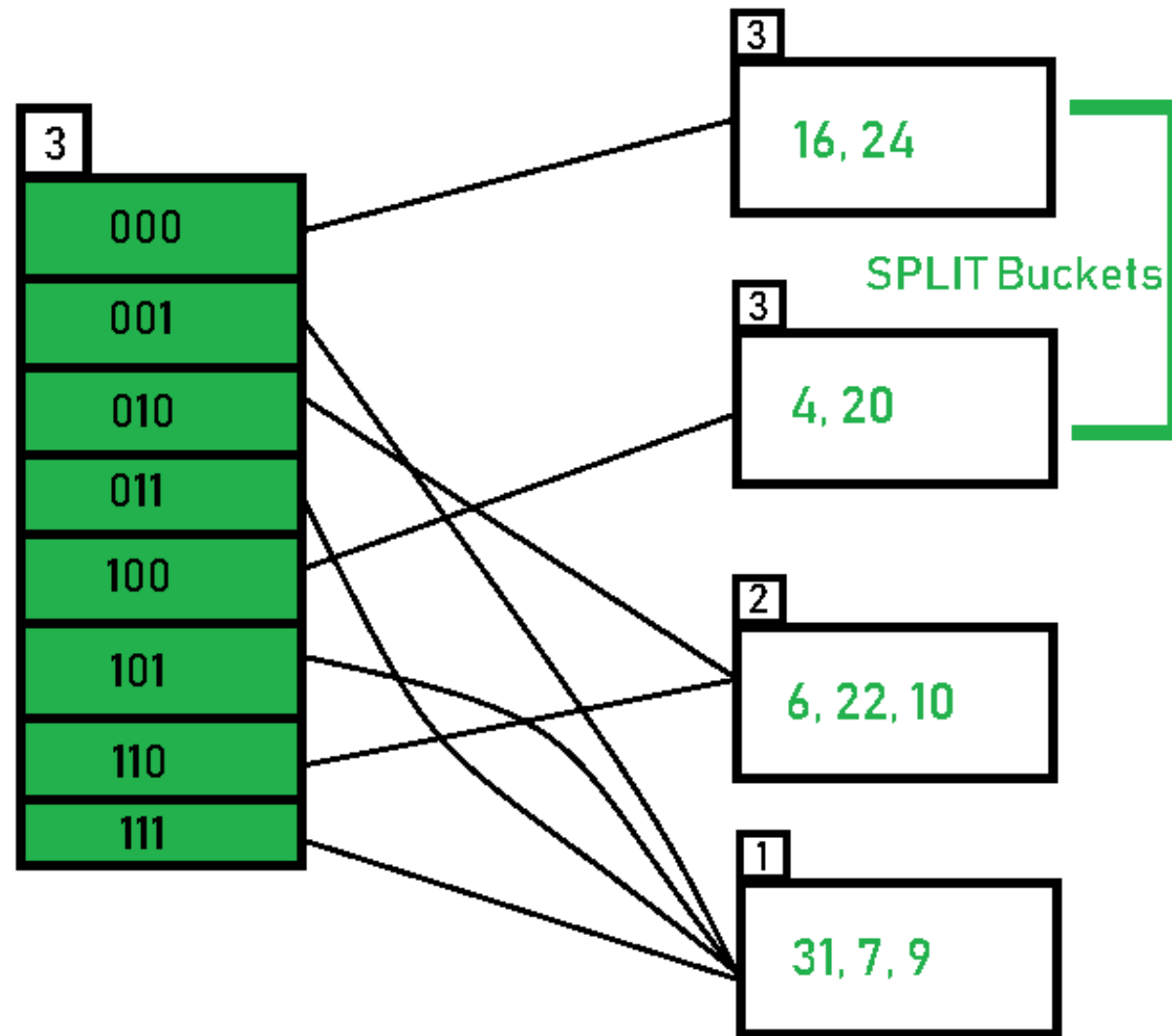
EXTENDIBLE HASHING – EXAMPLE

0 is inserted in bucket pointed out by 00.

Case 1, since the local depth of the bucket = global-depth, directory expansion (doubling) takes place along with bucket splitting.

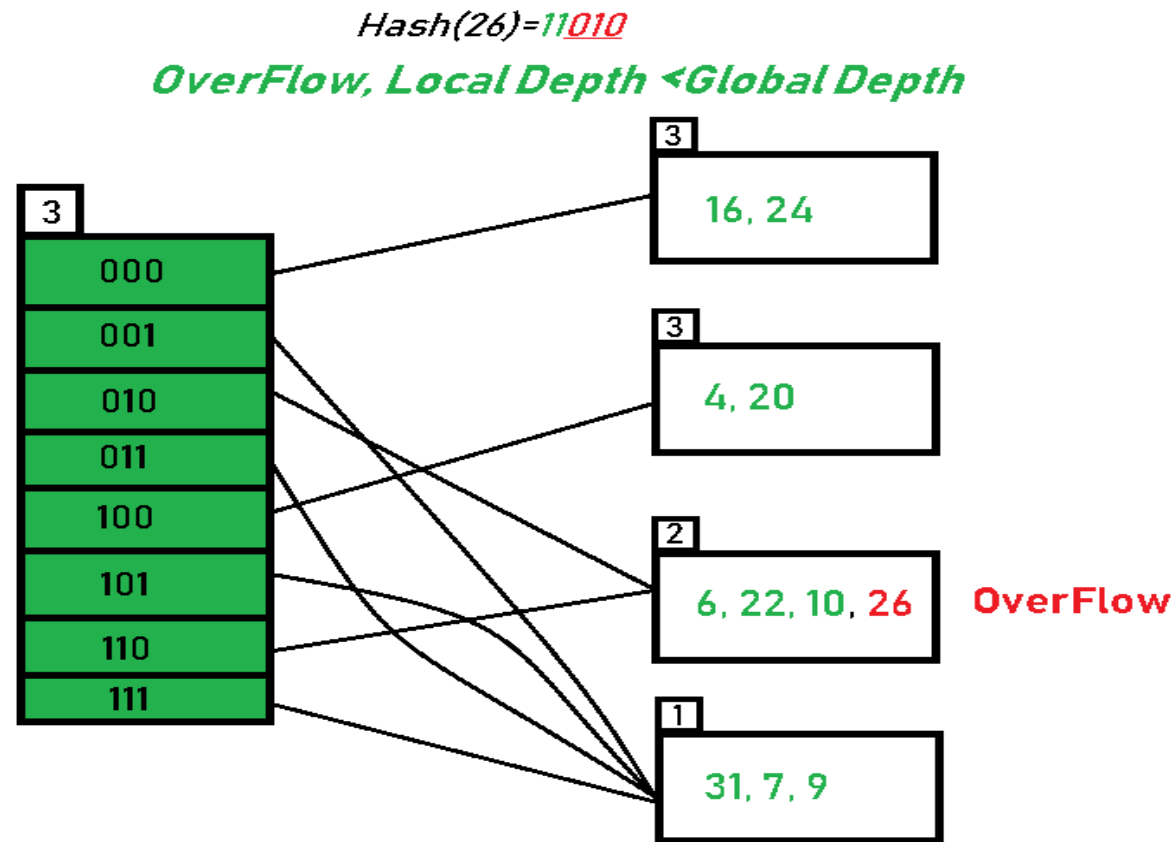
Elements present in overflowing bucket are rehashed with the new global depth.

EXTENDIBLE HASHING – EXAMPLE



EXTENDIBLE HASHING – EXAMPLE

- ✓ Inserting 26: Global depth is 3. Hence, 3 LSBs of 26(11010) are considered. Therefore 26 best fits in the bucket pointed out by directory 010.

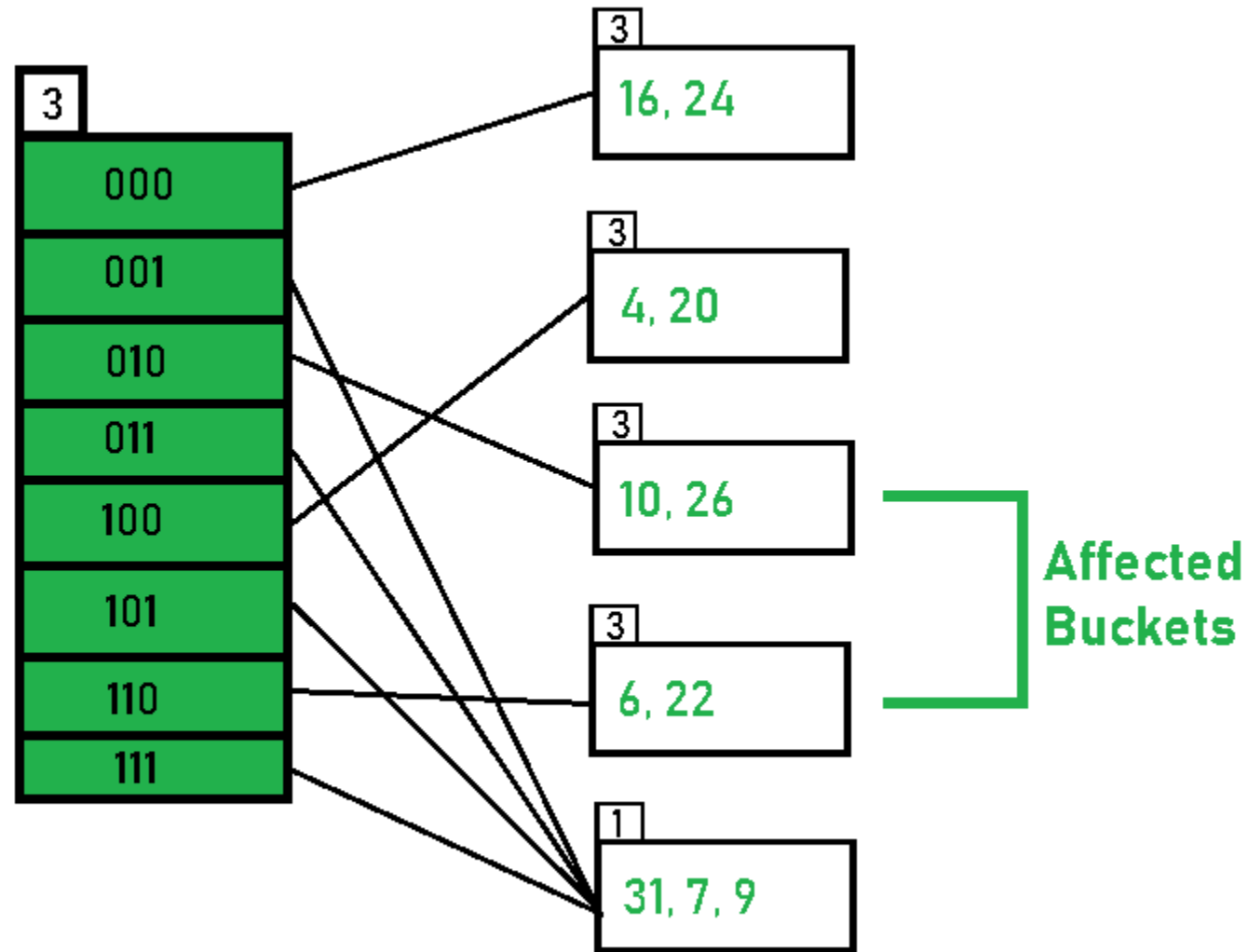


EXTENDIBLE HASHING – EXAMPLE

- ✓ The bucket overflows
- ✓ Case 2, since the local depth of bucket $<$ Global depth ($2 < 3$), directories are not doubled but only the bucket is split, and elements are rehashed.
- ✓ Finally, the output of hashing the given list of numbers is obtained.

EXTENDIBLE HASHING – EXAMPLE

Hashing of 11 Numbers is Thus Completed



EXTENDIBLE HASHING – POINTS

- ✓ A Bucket will have more than one pointers pointing to it if its local depth is less than the global depth.
- ✓ When overflow condition occurs in a bucket, all the entries in the bucket are rehashed with a new local depth.
- ✓ The **size of a bucket cannot be changed after the data insertion process begins**

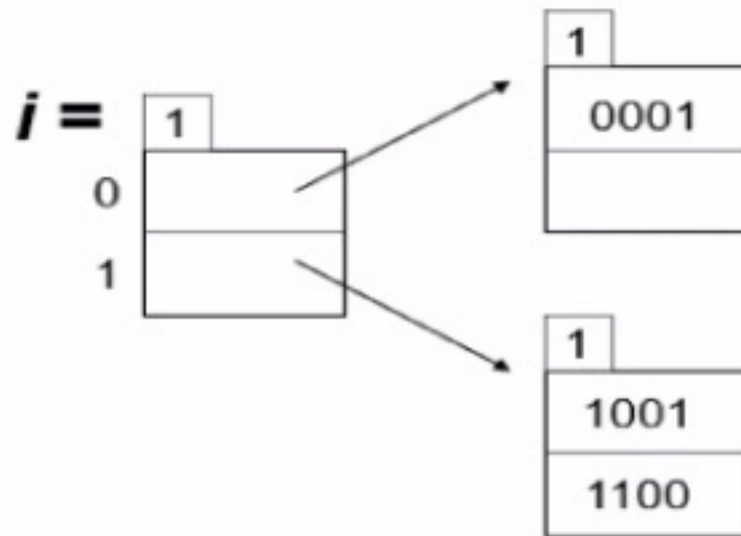
EXTENDIBLE HASHING – POINTS

- ✓ There is an array of Pointers to buckets
- ✓ The array of pointers can grow. Its length is always a power of 2. so, in a growing step the number of buckets doubles.
- ✓ There doesn't have to be a data block for each bucket. Certain buckets can share a block if the total number of records in those buckets can fit in the block
- ✓ The has function h computes for each key a sequence of k bit for some large k .
- ✓ The bucket numbers will at all times use some smaller number of bits say l bits from the beginning of this sequence of k bits. i.e., 2^l maximum no. of entries.
- ✓ The number $[j]$ appearing in the nub of each buckets, indicates how many bits of the hash function's sequence is used to determine membership of records in this block

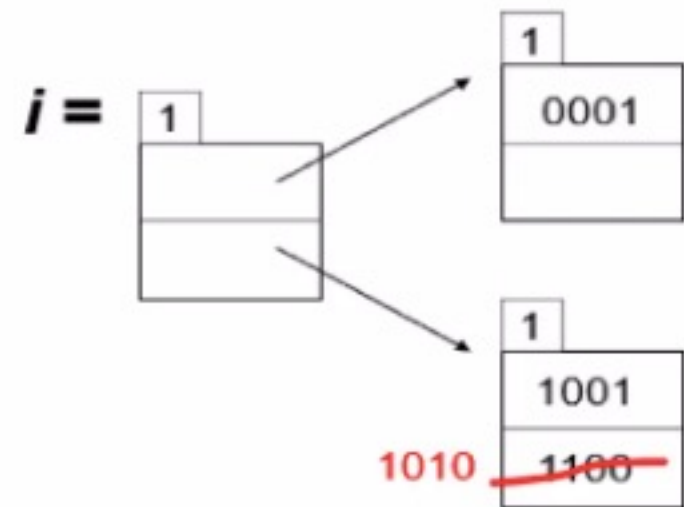
EXTENDIBLE HASHING – POINTS

Insertion:

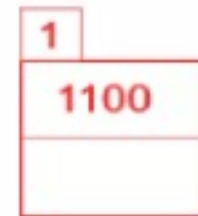
- ▶ Inserting 1010



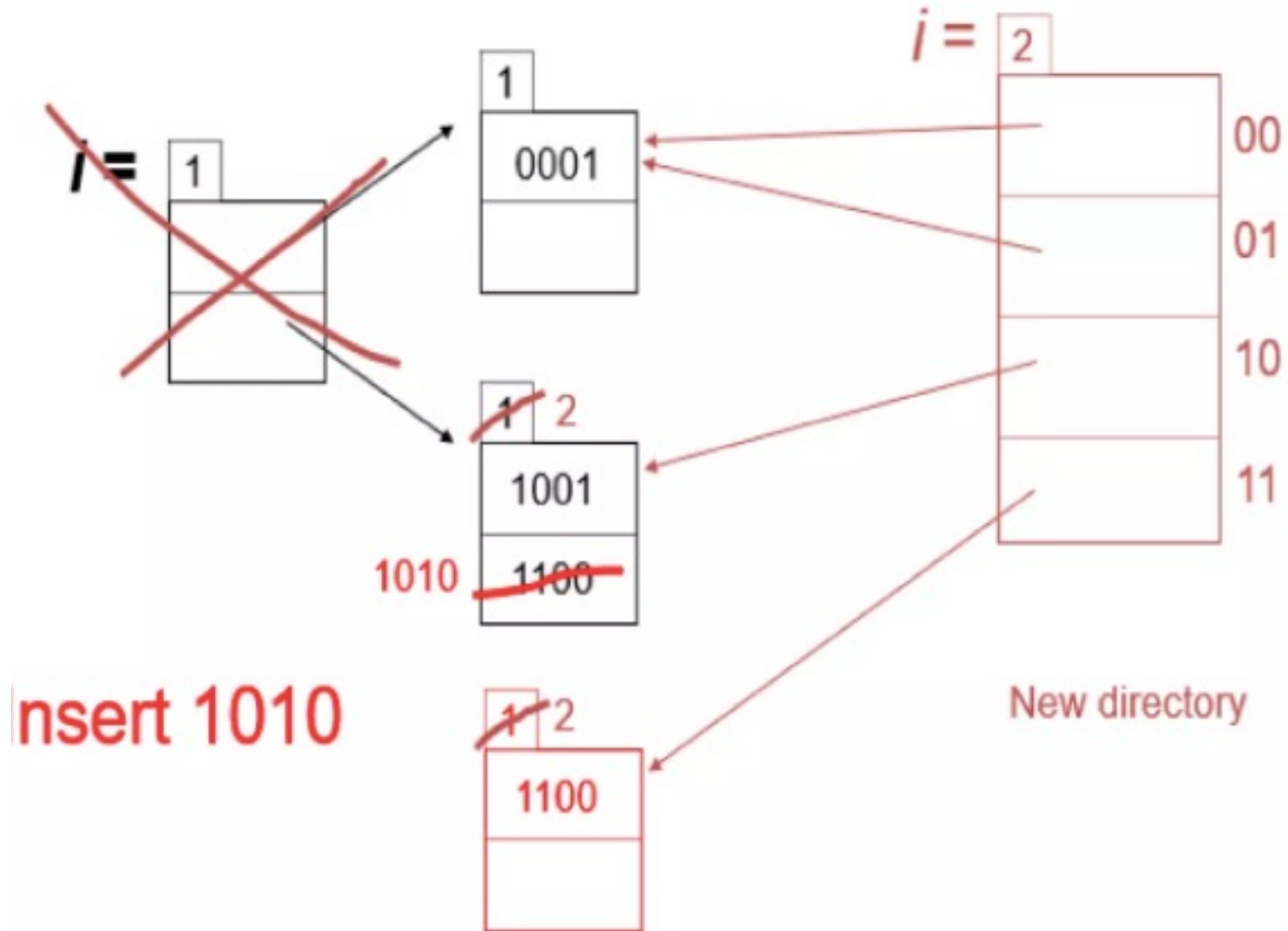
Insert 1010



Insert 1010



EXTENDIBLE HASHING – POINTS



EXTENDIBLE HASHING – POINTS

- ✓ Extendable hashing is a flexible, dynamic hashing system.
- ✓ An extendible hash table (EHT) has two components:
 - ✓ Directories
 - ✓ Buckets
- ✓ **Directories:** The directories of extendible hash tables store pointers to buckets. The number of directories of an EHT is referred to as the global depth of the EHT.

EXTENDIBLE HASHING – POINTS

- ✓ **Buckets:** Used to **hash the actual data**. **local depth** is used to decide the action that to be performed in case an overflow occurs.
- ✓ **Relation between Directories and Buckets:** A multiple directories can point to the same bucket.
- ✓ No. of Directories Entry = $2^{\text{global Depth}}$.
- ✓ Initial no. of buckets = $2^{\text{local Depth}}$
- ✓ Always Global Depth $\geq$ Local Depth.

EXTENDIBLE HASHING – OVERVIEW

- ✓ An EHT allows for a dynamic restructuring of the underlying structure.
- ✓ As the number of elements inserted into a hash table increases, so does the likelihood of collisions or overflow.
- ✓ Overhead can be reduced by taking preemptive action as the structure grows.
- ✓ The preemptive action consists of splitting the contents of given bucket and increasing the number of buckets.

EXTENDIBLE HASHING – OVERVIEW

- ✓ The number of LSB's used is equivalent to the number of bits needed to index all of the directories.
- ✓ if there are two directories, a single bit can index all of the directories. If, however, there are four directories, two bits will be needed to index them.

EXTENDIBLE HASHING – ALGORITHM

- ✓ Initialize the bucket depths and the global depth of the directories.
- ✓ Convert data into a binary representation.
- ✓ Consider the "global depth" number of the least significant bits (LSBs) of data.
- ✓ Map the data according to the ID of a directory.
- ✓ Check for the following conditions if a bucket overflows (if the number of elements in a bucket exceeds the set limit):
 - ✓ Global depth == bucket depth: Split the bucket into two and increment the global depth and the buckets' depth. Re-hash the elements that were present in the split bucket.
 - ✓ Global depth > bucket depth: Split the bucket into two and increment the bucket depth only. Re-hash the elements that were present in the split bucket.
- ✓ Repeat the steps above for each element.

EXTENDIBLE HASHING – EXAMPLE

- ✓ Let's take the following example to see how this hashing method works where:
- ✓ Data = {28,4,19,1,22,16,12,0,5,7}
- ✓ Bucket limit = 3

EXTENDIBLE HASHING – EXAMPLE

✓ Convert the data into binary representation:

✓ $28 = 11100$

✓ $4 = 00100$

✓ $19 = 10011$

✓ $1 = 00001$

✓ $22 = 10110$

✓ $16 = 10000$

✓ $12 = 01100$

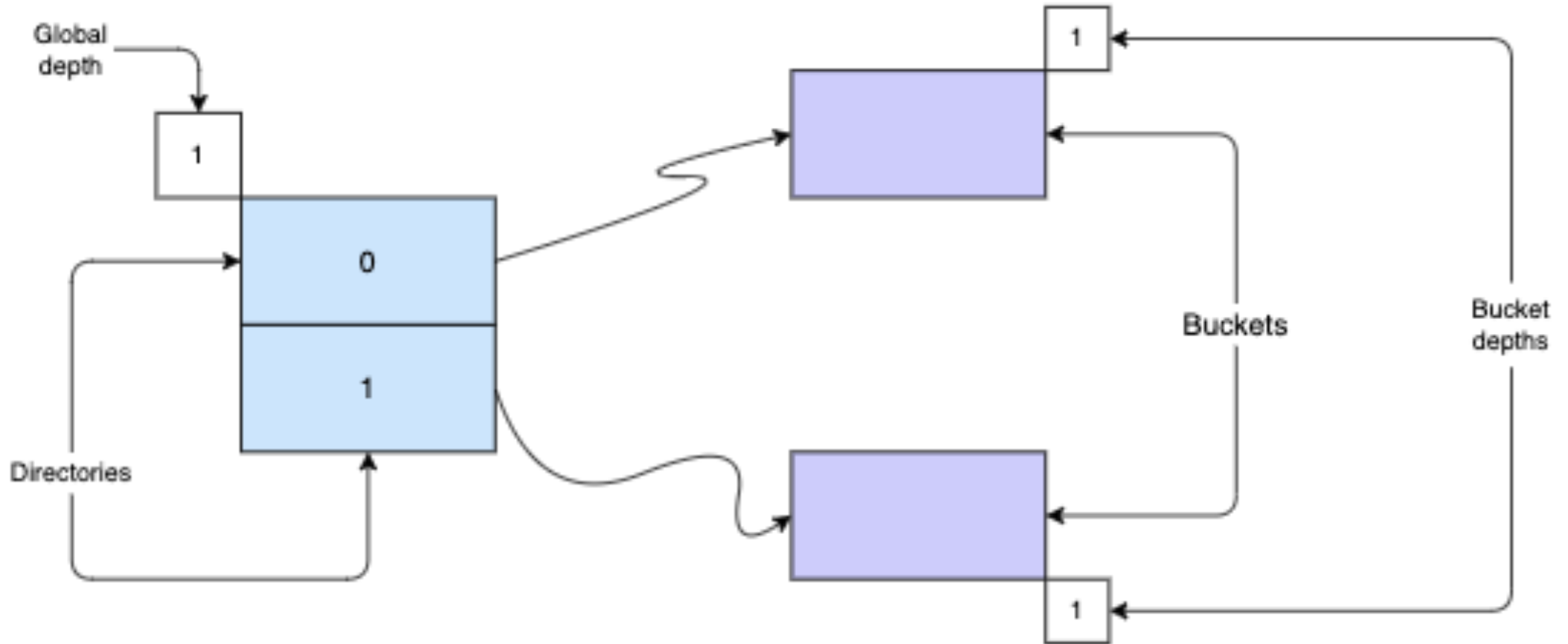
✓ $0 = 00000$

✓ $5 = 00101$

✓ $7 = 00111$

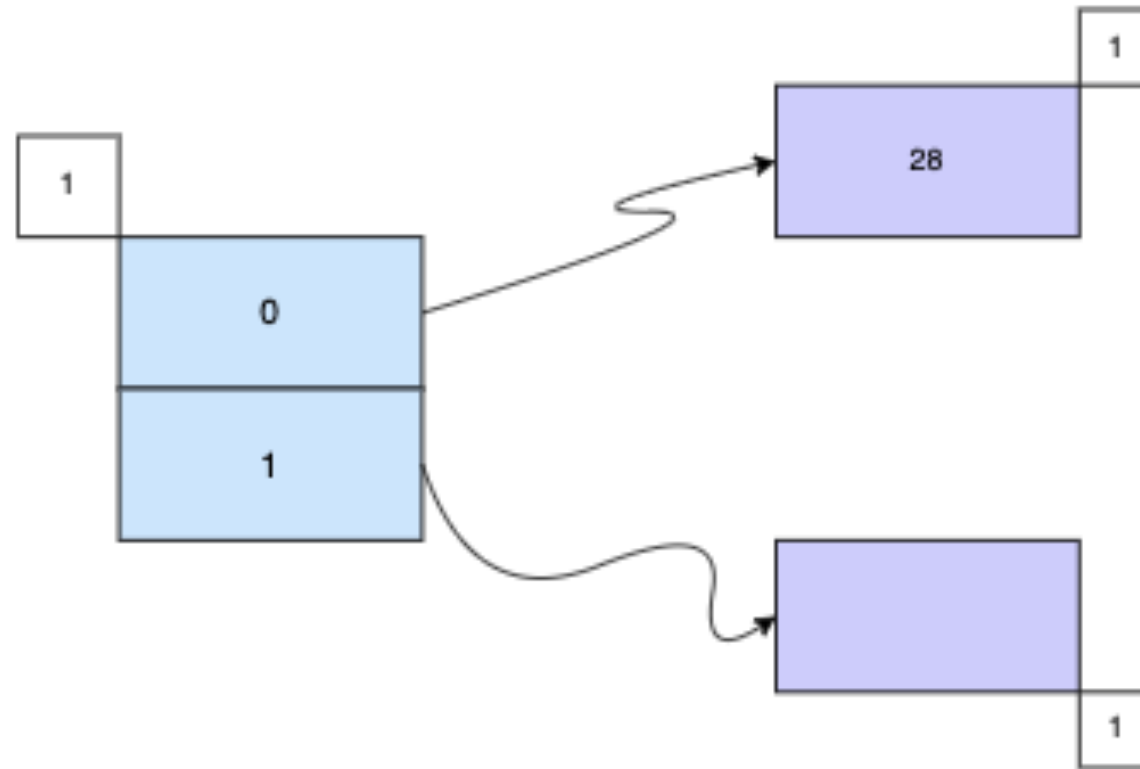
EXTENDIBLE HASHING – EXAMPLE

- ✓ Initialize the hash table with two initial directories and buckets.
- Set the global depth and bucket depth to 1



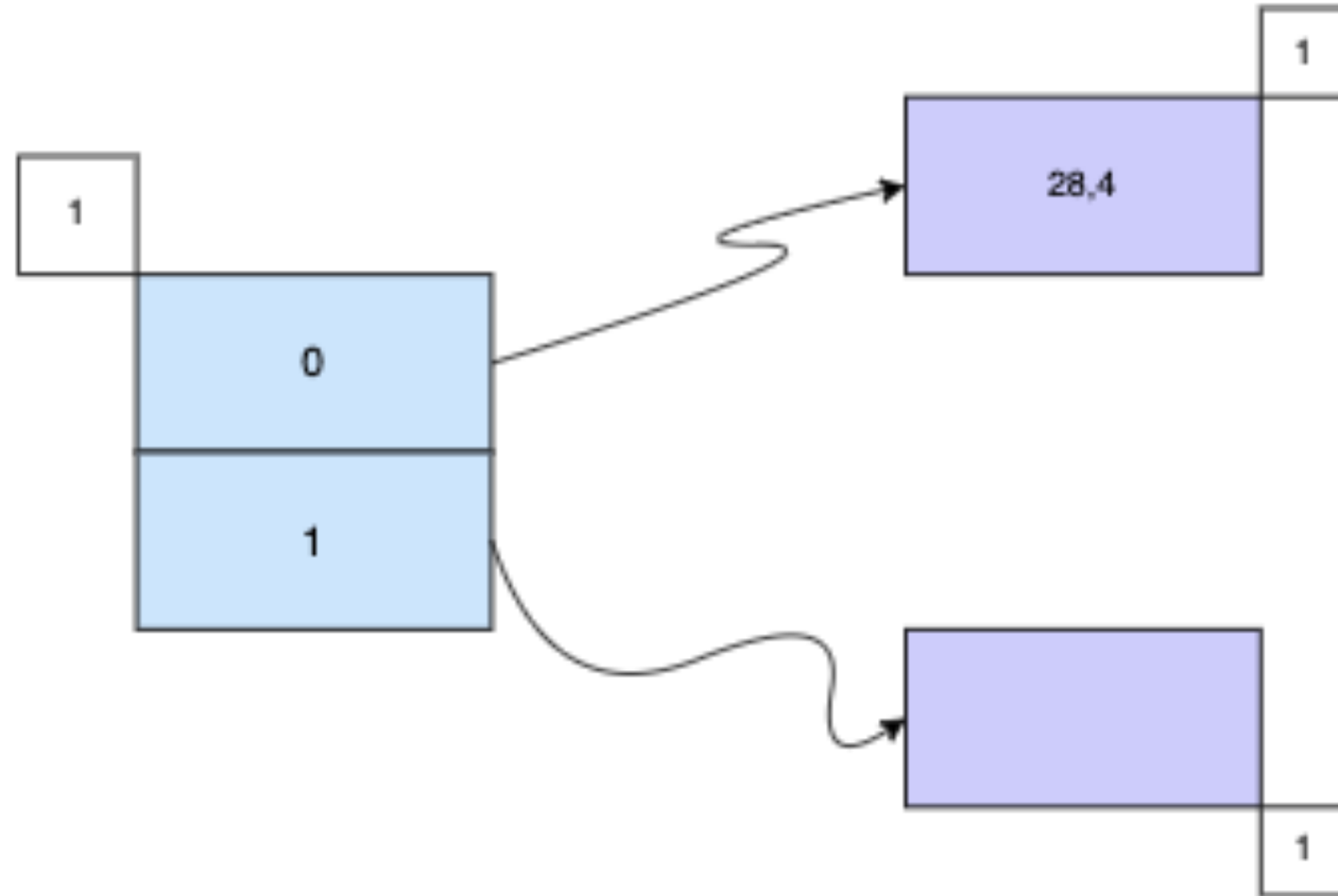
EXTENDIBLE HASHING – EXAMPLE

- ✓ Consider $28 = 11100$. Take the global depth number of LSBs, which is currently 1. We consider 0 as it is the rightmost LSB in $1110\textcolor{red}{0}$, and 28 is placed in the "0" ID bucket.



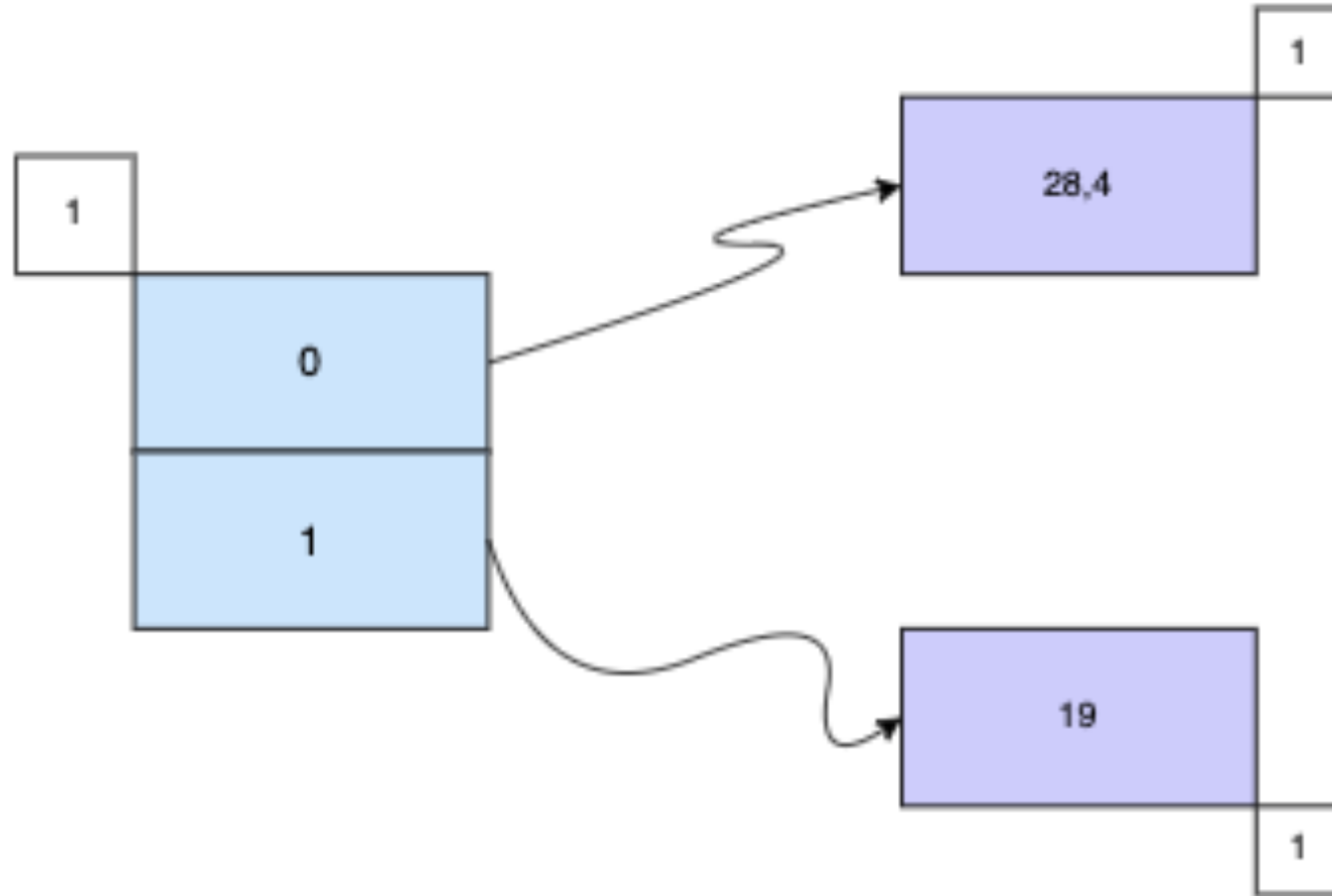
EXTENDIBLE HASHING – EXAMPLE

- ✓ Consider $4 = 00100$. Add it to the "0" ID bucket as one LSBs will be considered.



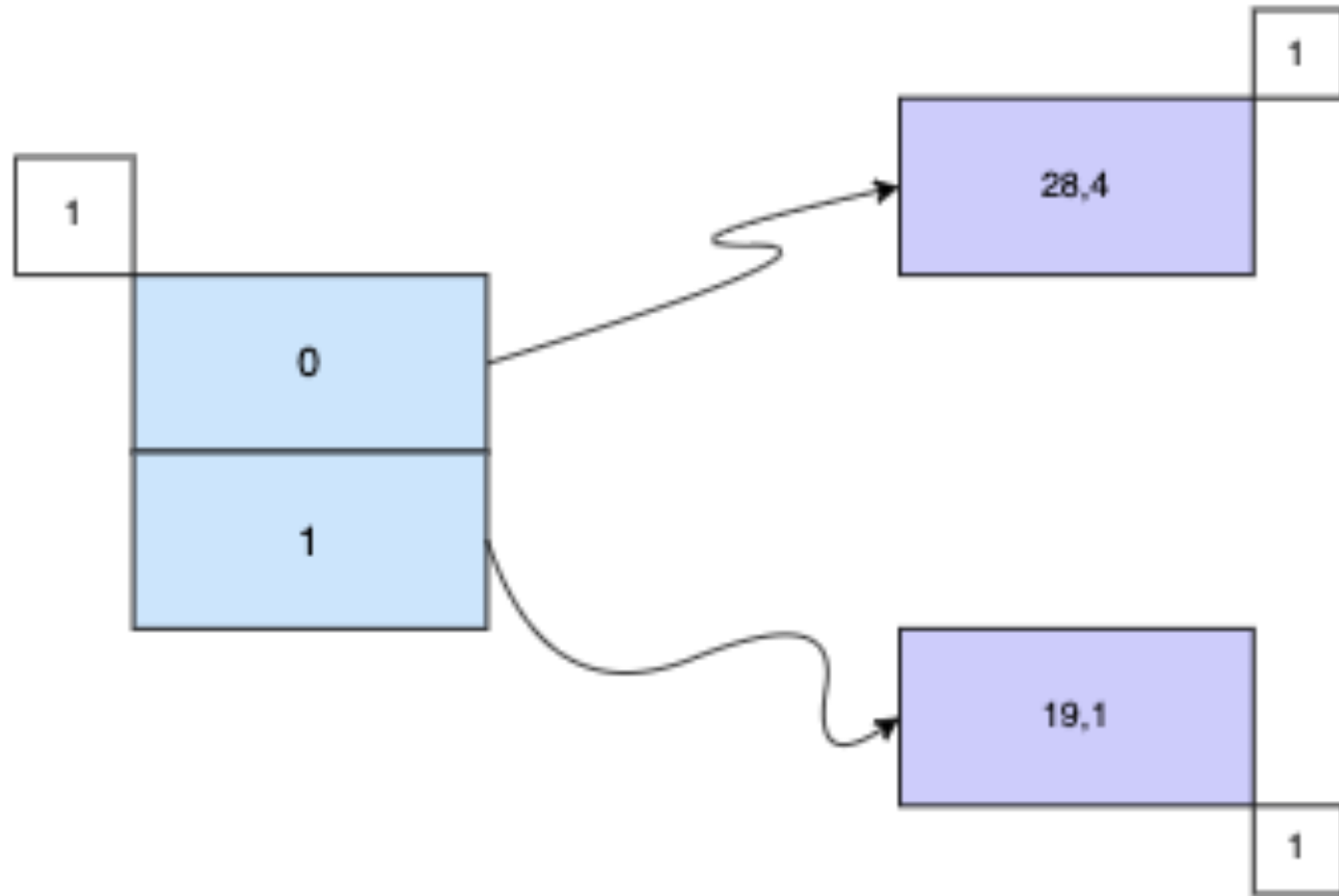
EXTENDIBLE HASHING – EXAMPLE

- ✓ Consider $19 = 10011$. Add it to the "1" ID bucket as one LSBs will be considered.



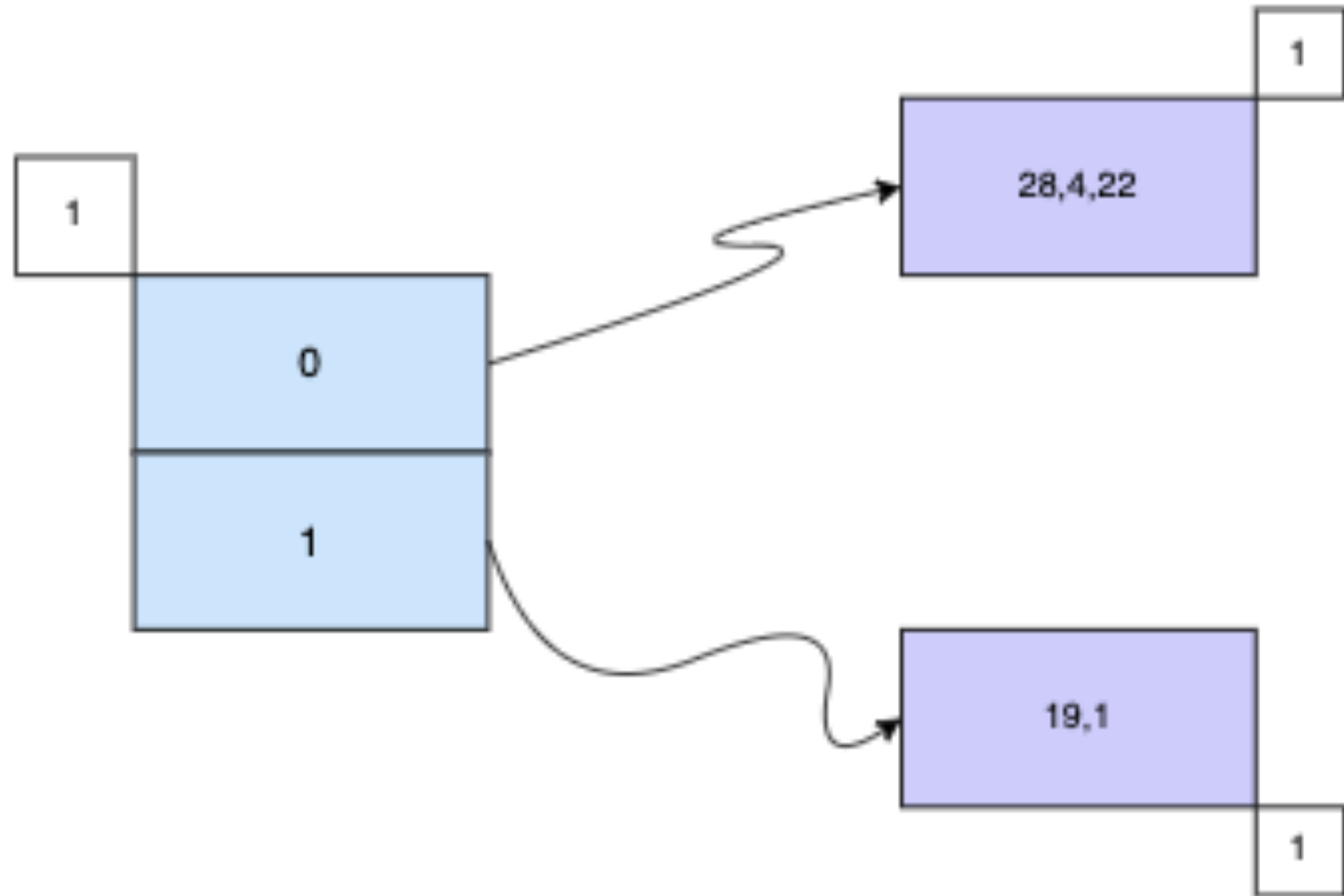
EXTENDIBLE HASHING – EXAMPLE

- ✓ Consider $1 = 00001$. Add it to the "1" ID bucket as one LSBs will be considered



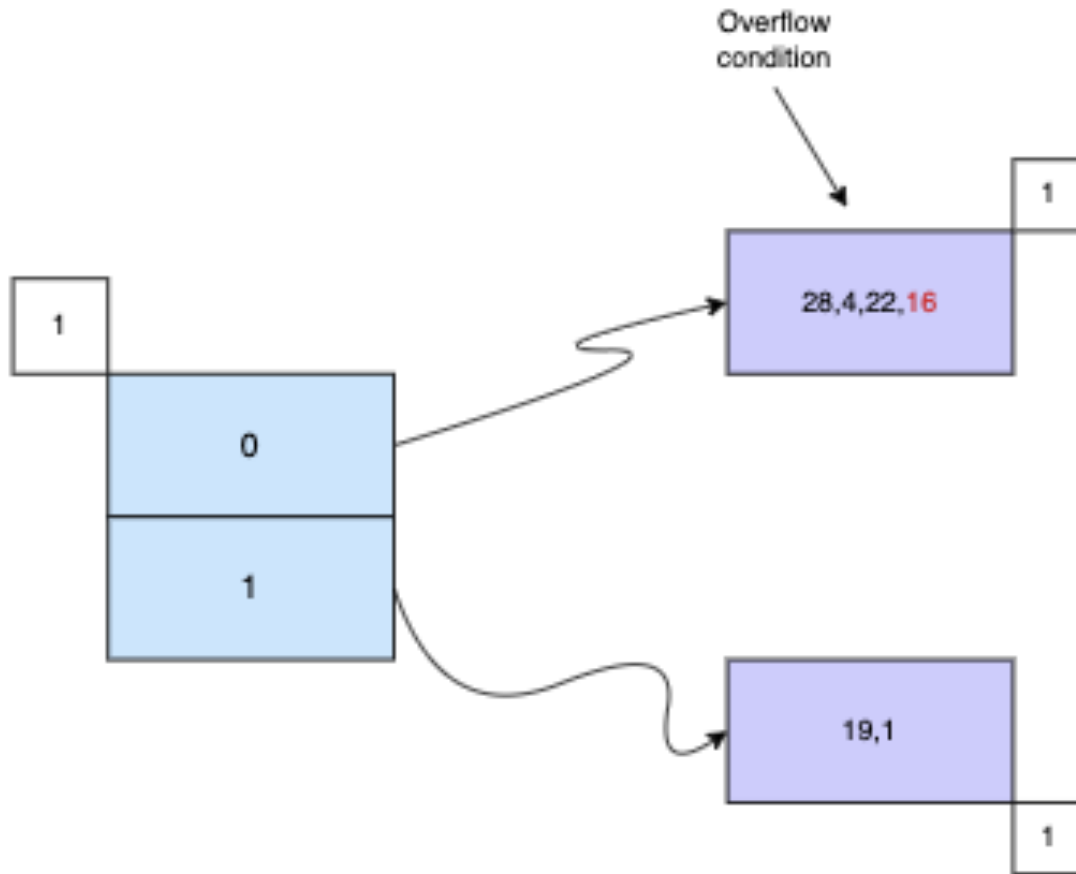
EXTENDIBLE HASHING – EXAMPLE

- ✓ Consider $22 = 10110$. Add it to the "0" ID bucket as one LSBs will be considered.



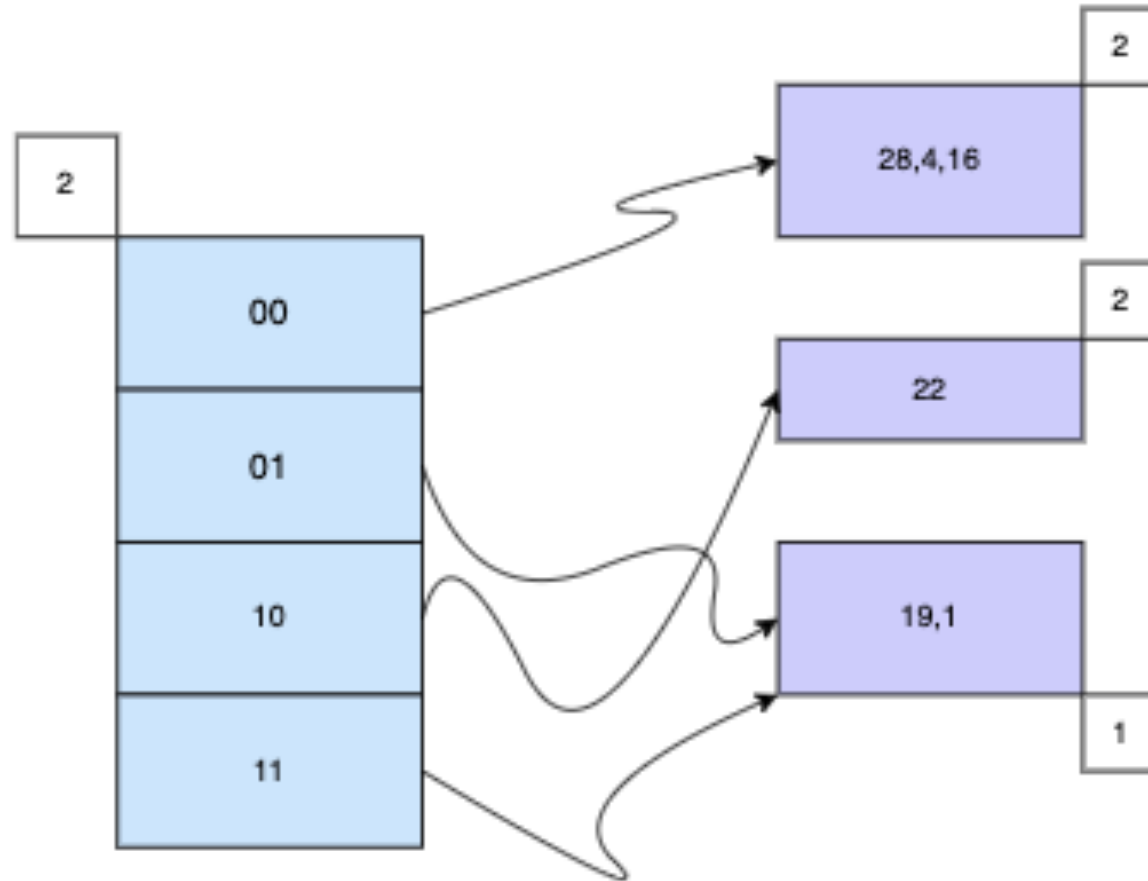
EXTENDIBLE HASHING – EXAMPLE

- ✓ Consider $16 = 10000$. Adding it to the "0" ID bucket makes it overflow, and the first condition is satisfied where global depth == bucket depth.



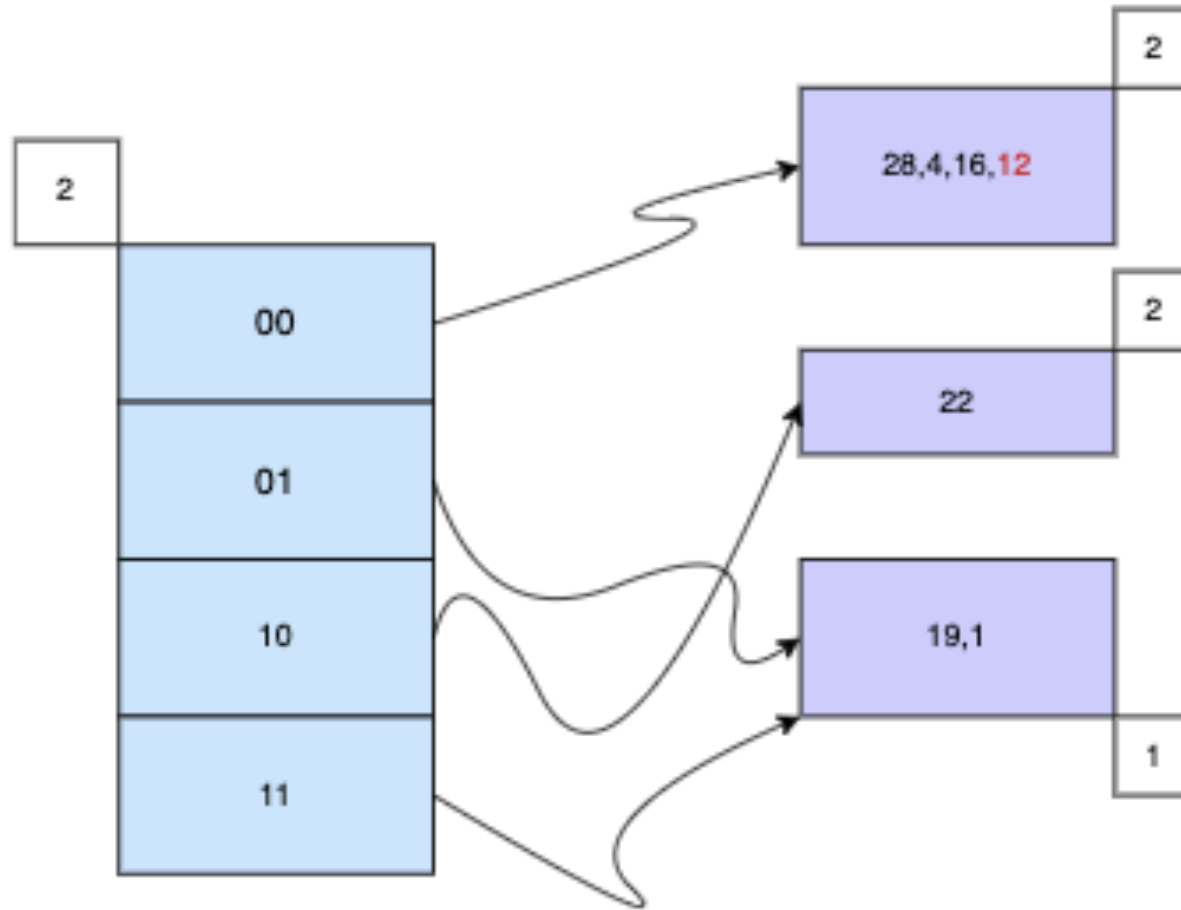
EXTENDIBLE HASHING – EXAMPLE

- ✓ Split the bucket, grow directories, increment the global and bucket depths, and re-hash the bucket elements according to two LSBs.



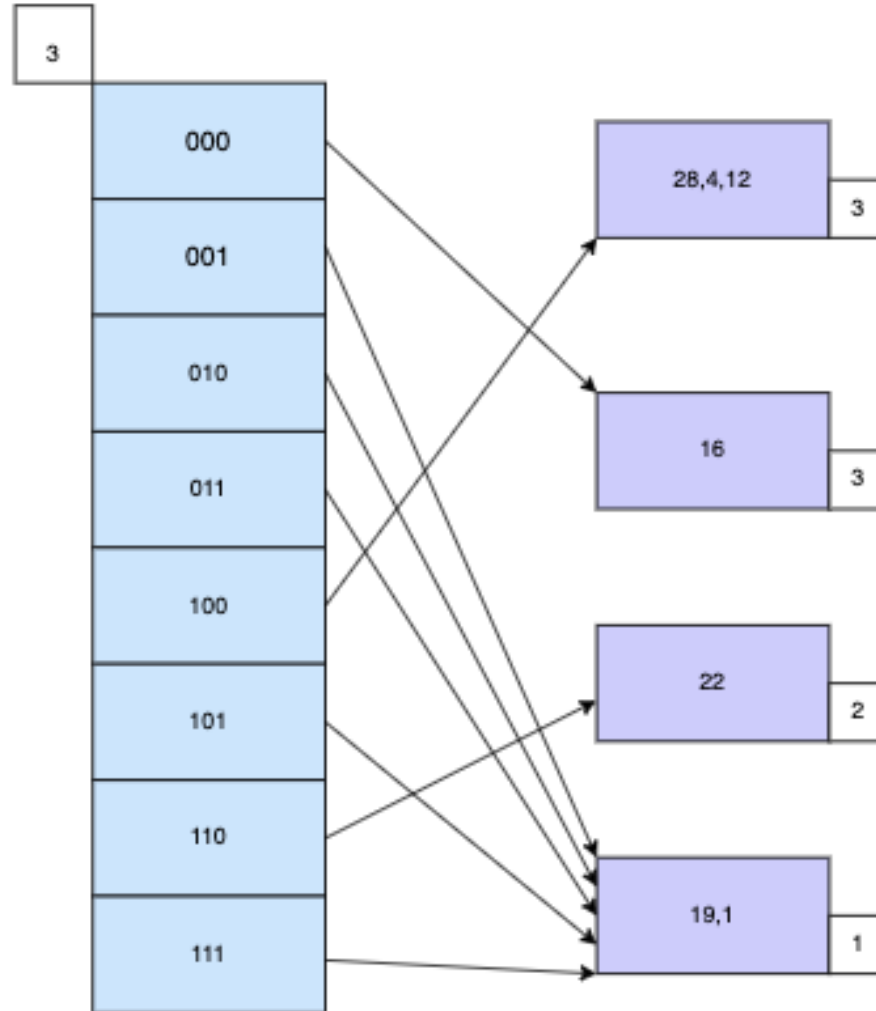
EXTENDIBLE HASHING – EXAMPLE

- ✓ Consider $12 = 01100$. Adding it to the "00" ID bucket according to two LSBs makes it overflow, and satisfies the first condition.



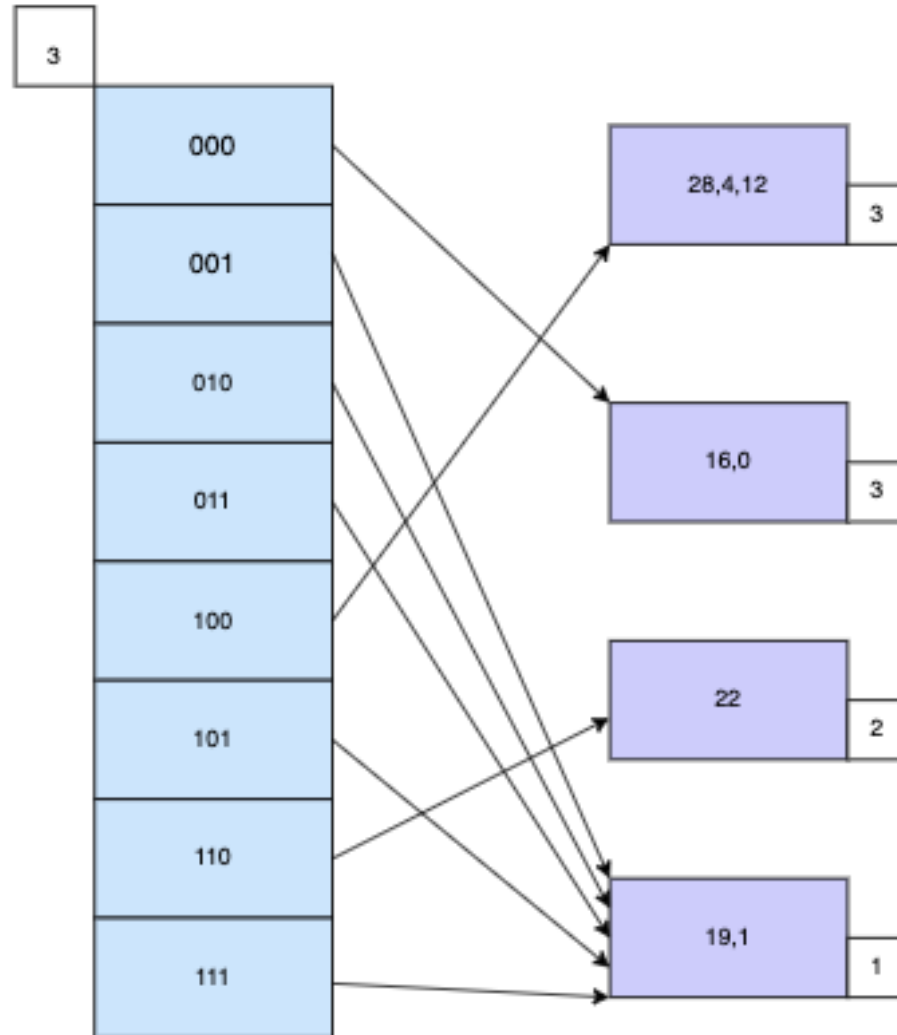
EXTENDIBLE HASHING – EXAMPLE

- ✓ Repeat the previous step and re-hash the elements of the bucket according to three LSBs.



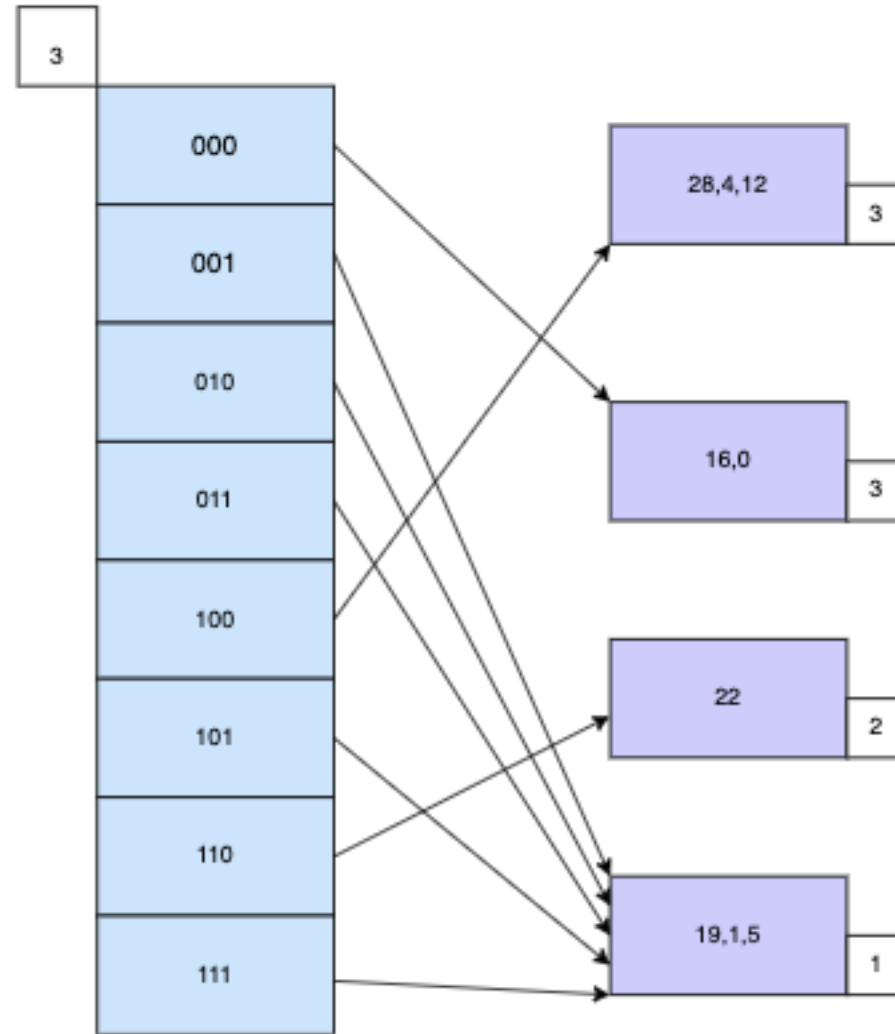
EXTENDIBLE HASHING – EXAMPLE

- ✓ Consider $0 = 00000$. Add it to the "000" ID bucket as three LSBs are considered.



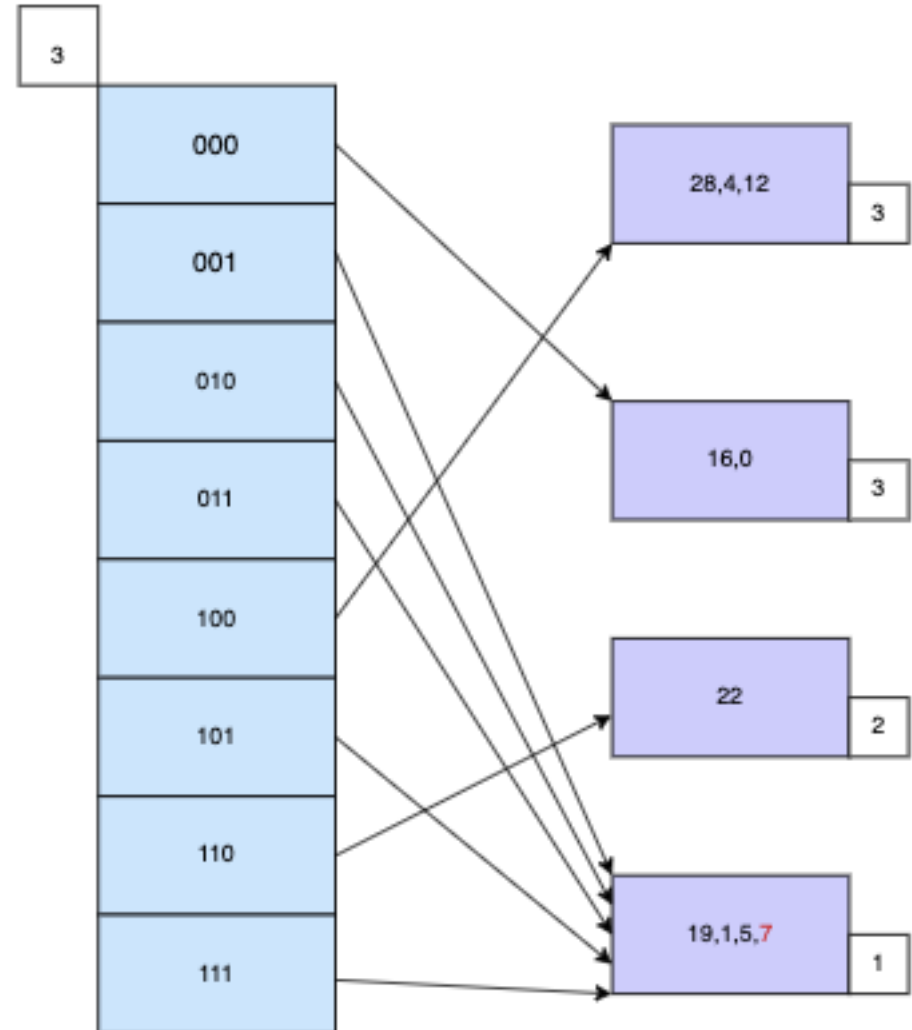
EXTENDIBLE HASHING – EXAMPLE

- ✓ Consider $5 = 00101$. Add it to the "101" bucket as three LSBs are considered



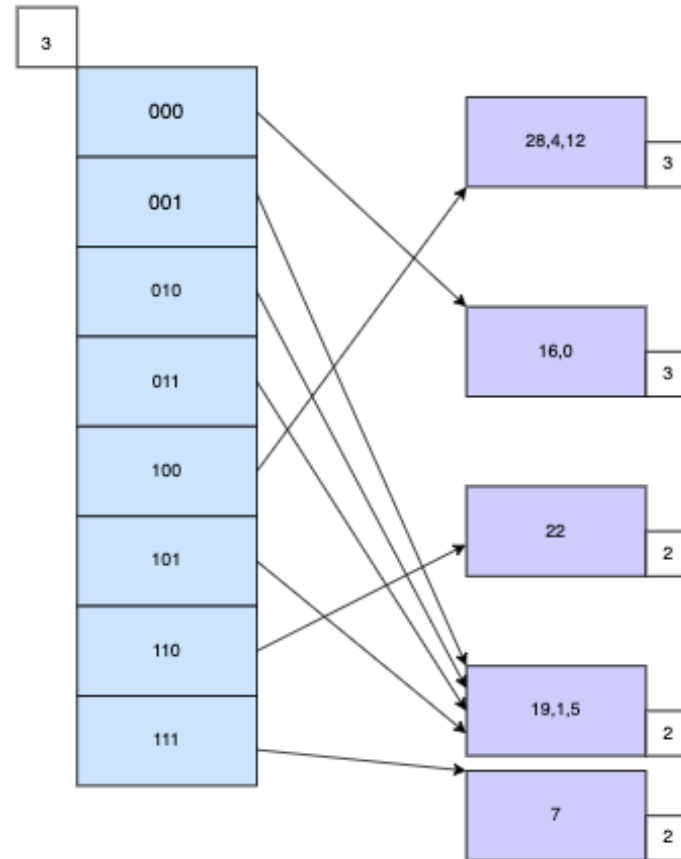
EXTENDIBLE HASHING – EXAMPLE

- ✓ Consider $7 = 00111$. Adding it to the "111" bucket according to three LSBs makes it overflow



EXTENDIBLE HASHING – EXAMPLE

- ✓ The bucket depth is now less than global depth. So, the second condition is fulfilled, and now only the bucket split step is executed. Elements are re-hashed according to the global depth, which is still 3.



RANDOM PROBING - HASHING

- ✓ Hash fun $H(x) = (\text{key mod } 11) + 1$
- ✓ When collision occurs
- ✓ **$H(x) = (i+p)\% n + 1$**
 - ✓ where i is the index value and
 - ✓ p is the prime no, (This can be a random $0 < \text{prime no.} < n$)
 - ✓ n is the size of the hash table

RANDOM PROBING – HASHING - EXAMPLE

✓ Insert 23

✓ $i = (23 \% 11) + 1 = 2$

✓ Insert 45

✓ $i = (45 \% 11) + 1 = 2$

✓ collision occurs

0	
1	
2	23
3	
4	
5	
6	
7	
8	
9	
10	

RANDOM PROBING – HASHING - EXAMPLE

✓ When collision (Here prime no. 5 has been chosen randomly)

$$i = (i+P)\%11+1$$

$$=(2+5)\%11+1=8$$

$$=(8+5)\%11+1=3$$

$$=(3+5)\%11+1=9$$

$$=(9+5)\%11+1=4$$

$$(4+5)\%11+1=10$$

$$(10+5)\%11+1=5$$

$$(5+5)\%11+1=11$$

$$(11+5)\%11+1=6$$

0	
1	
2	23
3	56
4	78
5	
6	
7	
8	45
9	67
10	89

RANDOM PROBING – HASHING - EXAMPLE

- ✓ A hash table of size 7 and a simple hash function that returns the key modulo 7. We want to insert the keys 10, 22, 31, 4, and 15 into the hash table.

0	
1	
2	
3	
4	
5	
6	
7	

RANDOM PROBING – HASHING - EXAMPLE

✓ Insert 10, 22, 31:

$$10 \% 7 = 3$$

$$22 \% 7 = 1$$

$$31 \% 7 = 3$$

Collision Occurred

Now use this to find a random probe sequence.

$$\mathbf{H(x) = (i+p)\% n + 1}$$

$$= (3 + 5) \% 7 + 1 = 2$$

$$= (2 + 5) \% 7 + 1 = 1$$

0	
1	22
2	
3	10
4	
5	
6	
7	

RANDOM PROBING – HASHING - EXAMPLE

✓ Insert 31: Collision Occurred

$$H(x) = (i + p) \% n + 1$$

$$= (3 + 5) \% 7 + 1 = 2$$

$$= (2 + 5) \% 7 + 1 = 1$$

$$= (1 + 5) \% 7 + 1 = 7$$

$$= (7 + 5) \% 7 + 1 = 5$$

$$= (5 + 5) \% 7 + 1 = 4$$

$$= (4 + 5) \% 7 + 1 = 3$$

0	
1	22
2	31
3	10
4	
5	
6	
7	

Check from first index which one is empty. 2 is empty so insert 31 there

RANDOM PROBING – HASHING - EXAMPLE

✓ Insert 4, 15:

$$4 \% 7 = 4$$

$15 \% 7 = 1$ Collision Occurred.

Find random index using

$$\mathbf{H(x) = (i+p)\% n + 1}$$

$$= (1 + 5) \% 7 + 1 = 7$$

7 is empty hence enter 15 there.

0	
1	22
2	31
3	10
4	4
5	
6	
7	15